

Deep learning enabled intrusion detection system for Industrial IOT environment

Himanshu Nandanwar^{*}, Rahul Katarya

Department of Computer Science and Engineering, Delhi Technological University, New Delhi, India

ARTICLE INFO

Keywords:

Cyber security
Deep learning
Dependability
Industrial Internet of Things (IIoT)
Intrusion detection system (IDS)
Privacy

ABSTRACT

The prevalence of security vulnerabilities in Internet of Things (IoT) applications poses a serious threat to enterprise systems, necessitating sophisticated and reliable defense solutions to counter emerging and evolving threats. For the Industrial Internet of Things (IIoT), stakeholders require trustworthy and sustainable systems that can prevent the loss of human life during critical operations. The impact of multi-variant persistent and sophisticated bot attacks on connected IIoTs is potentially catastrophic, and their detection presents a highly complex and critical challenge. Therefore, there is a pressing need for efficient and timely detection of IIoT botnet attacks. This research paper proposes a robust deep learning model named AttackNet for the detection and classification of different botnet attacks in IIoT based on adaptive based CNN-GRU model. The model is extensively evaluated using the latest dataset and standard performance evaluation metrics, demonstrating its capacity to protect IIoT networks against sophisticated cyber-attacks with a testing accuracy of 99.75%, a loss of 0.0063, precision and recall score of 99.75% and 99.74% respectively. Our proposed model demonstrates superior accuracy, particularly within the N_BaIoT dataset. It achieves an outstanding accuracy of 99.75% across ten classes, surpassing state-of-the-art techniques by a substantial margin ranging from 3.2% to 16.07%. Moreover, the proposed model outperforms state-of-the-art anomaly detection systems in IIoT based on a real-time IoT device dataset in terms of detecting and classifying botnet attacks accurately.

1. Introduction

The Internet of Things (IoT) has revolutionized the information industry and transformed people's lives. IoT aims to seamlessly merge the physical and digital worlds, creating a unified ecosystem that ushers in a new era of intelligent connectivity (Sha, Wei Wei, Andrew Yang, Zhiwei Wang, & Weisong Shi, 2018). Security is a critical aspect of IoT, as its inherent openness makes it vulnerable to a wide range of security threats. The diverse nature of IoT and the constant evolution of attacks and threats make traditional security defense architectures inefficient in effectively countering various uncertainties (Hassan, Rosilah, Faizan Qamar, Mohammad Kamrul Hasan, Azana Hafizah Mohd Aman, & Amjed Sid Ahmed, 2020). As IoT finds applications in various fields such as Industrial IoT (IIoT), the Internet of Vehicles (IoV), smart medical systems, smart homes, and smart grids, the number and complexity of IoT applications and their centralized data management platforms have increased. This is shown in Fig. 1. These intricate circumstances have exposed security issues that impede the progress of IoT (Lv, Zhihan and Qiao, Liang and Li, Jinhua and Song, & Houbing, 2021). Failure to

ensure the security of IoT systems can result in the theft or exploitation of personal information, trade secrets, and even military intelligence, significantly compromising personal privacy, economic stability, national security, and military interests. Consequently, an urgent need exists to address IoT security concerns and establish new security mechanisms tailored to the characteristics and application domains of IIoT, effectively safeguarding the IIoT ecosystem. The importance of protecting IIoT devices and networks cannot be overstated.

In this research, we aimed to identify techniques for securing IIoT environments and understanding the various scenarios and vulnerabilities that exist in IIoT applications (Elrawy, Awad, & Hamed, 2018). The fundamental principles of IIoT security must be taken into account to establish an effective security system, these primary elements of IIoT security are:

- Confidentiality, which involves protecting sensitive information from unauthorized access. IIoT devices often collect and store sensitive information, such as personal data, patient records, and

^{*} Corresponding author.

E-mail addresses: himanshunandanwar9cm0@gmail.com (H. Nandanwar), rahuldtu@gmail.com (R. Katarya).

corporate information, which must be safeguarded against unauthorized access (Mohamed Abomhara, & Geir, Koen, 2015).

- Integrity is another critical aspect of IoT security, as data is often transmitted over remote communication channels and must be protected from unauthorized modification. Ensuring data integrity requires a robust monitoring system that can detect any unauthorized modifications made during communication.
- Authentication is a vital component of IoT security, as it provides a mechanism for verifying the identity of devices and users. However, trade-offs must be considered when designing an authentication system, balancing security with flexibility.
- Authorization is also essential for IoT security, as it provides privileges to clients to access data and system resources. Ensuring the appropriate authorization for users and devices involved in IoT systems helps to maintain security and protect against unauthorized access (Oh, Se-Ra, Young-Gab Kim, & Sanghyun Cho, 2019).
- Availability is a crucial element of IoT security, as IoT devices must be continuously available to authorized users for administrative and communication tasks.
- Threats to availability, such as active jamming or denial-of-service attacks, can render devices unavailable and must be protected against to ensure the continued availability of IoT systems. Threats to IoT systems can be categorized as passive or active (Diro & Chiamkurti, 2018). Passive threats involve covert monitoring of communication channels to collect information, while active threats involve altering or manipulating IoT systems to control resources or disrupt availability (Kaur, & Singh, 2020). Denial-of-service attacks are a common example of active threats, which can be difficult to detect and prevent. IoT systems are particularly vulnerable to security threats due to their exponential interconnectivity and use of resource-constrained protocols. Proper security arrangements, such as detection mechanisms and secure communication systems, are necessary to ensure the practical and valuable implementation of IoT systems. IoT security is a complex and multifaceted issue that requires a comprehensive approach to protect against threats to

confidentiality, integrity, authentication, authorization, and availability. Users must be aware of the security flaws in IoT devices and networks to defend against cyber threats. Various techniques and structures, such as machine learning, can be employed to detect and classify attacks on extensive IoT networks.

1.1. Industrial Internet of Things (IIoT)

The Industrial Internet of Things (IIoT) is rapidly expanding and becoming an integral part of our daily lives. It is transforming various sectors, such as agriculture, healthcare, logistics, and transportation, by creating smart systems that generate large amounts of data (Iqbal, Rahat and Maniak, Tomasz and Doctor, Faiyaz & Karyotis Charalampos, 2019). However, this growth also exposes IIoT to various cyber threats, such as phishing, denial-of-service (DoS), man-in-the-middle (MITM), and botnets. These attacks aim to steal information, cause data loss, and compromise the entire system. Botnets are considered the most sophisticated and dangerous attacks that can propagate through networks and intelligent devices, exploiting vulnerabilities to gain remote access to cyber adversaries (Shevchik, Sergey and Masinelli, Giulio and Kenel, Christoph and Leinenbach, Christian & Wasmer Kilian, 2019). To protect the heterogeneous IIoT devices and their traffic, existing solutions rely on signature-based detection, which uses predefined signature vectors for pattern matching. However, this approach is insufficient for detecting zero-day threats, attacks, and vulnerabilities that arise due to the dynamic and heterogeneous nature of the IoT infrastructure. Therefore, deep learning (DL)-driven intelligent solutions are needed to enable zero-day threat detection and provide adaptive, resilient, reliable, and efficient botnet identification in IIoT. DL models can learn the patterns of normal traffic and identify anomalies that may indicate an attack. This approach is more robust than signature-based detection as it can detect new attacks that have not yet been seen before. Fig. 2 presents a visual representation of the five fundamental elements that contribute to the trustworthiness of Industrial Internet of Things (IIoT) systems

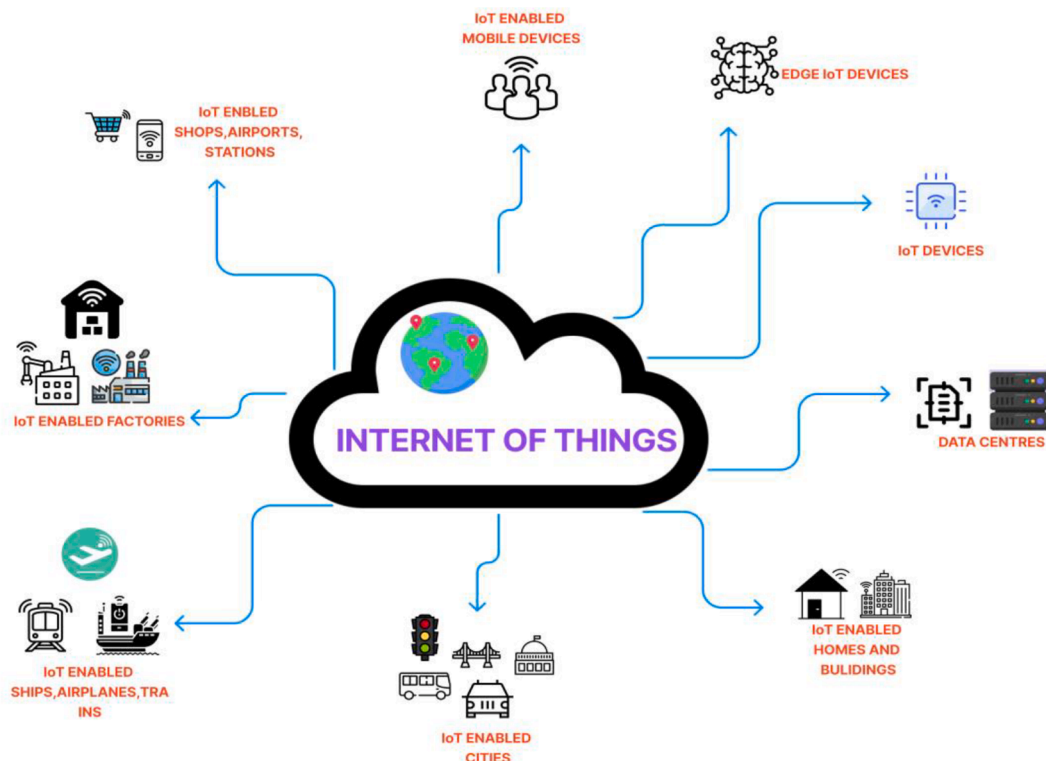


Fig. 1. Internet of Things mechanism: Applications, devices and data centers.

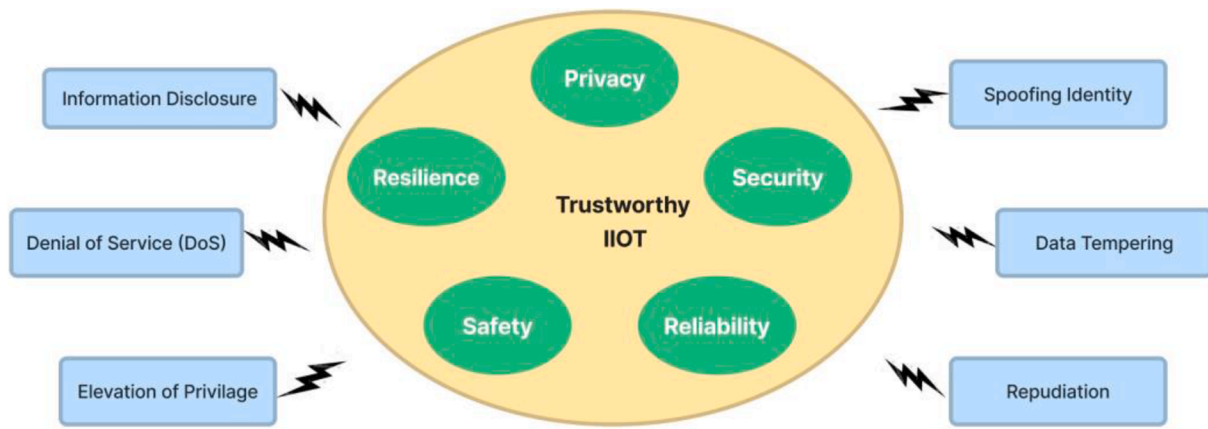


Fig. 2. Five pillar of IIoT trustworthiness.

(Fraile, Francisco and Tagawa, Takuya and Poler, Raul & Ortiz, Angel, 2018). Adhering to these parameters ensures the establishment of a secure and reliable IIoT platform, effectively mitigating various threats identified by the STRIDE threat model, such as spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege (Fraile, Francisco and Tagawa, Takuya and Poler, Raul & Ortiz, Angel, 2018).

1.2. Types of attack in IIoT environment

Industrial Internet of Things (IIoT) systems are vulnerable to a wide range of cyber-attacks. Here are some of the most common types of attacks that are possible in IIoT (Yasir Ali, Habib Ullah Khan, & Muhammad Khalid, 2023) (Sengupta, Sushmita Ruj, & Sipra Das Bit, 2020):

- **Malware attacks:** Malware is a type of software that is designed to harm or disrupt computer systems. Malware attacks can infect IIoT devices and cause them to malfunction or transmit sensitive data to attackers.
- **Denial-of-service (DoS) attacks:** DoS attacks are designed to overload IIoT systems with traffic or requests, making them unavailable to legitimate users. This can cause significant disruption to IIoT operations and may lead to downtime or loss of data.
- **Man-in-the-middle (MitM) attacks:** MitM attacks involve intercepting communication between IIoT devices and altering or stealing data. Attackers can use MitM attacks to steal sensitive data or gain unauthorized access to IIoT systems.
- **Phishing attacks:** Phishing attacks involve tricking users into revealing sensitive information, such as login credentials or financial data. Phishing attacks can be used to gain access to IIoT systems or steal sensitive.
- **Supply chain attacks:** Supply chain attacks involve targeting the vendors, manufacturers, or suppliers of IIoT devices or software to compromise the devices or software before deployment. This can allow attackers to gain access to IIoT systems or steal sensitive data.
- **Social engineering attacks:** Social engineering attacks involve using psychological manipulation to trick users into revealing sensitive information or performing actions that could compromise the security of IIoT systems. Examples of social engineering attacks include pretexting, baiting, and tailgating.
- **Zero-day attacks:** Zero-day attacks involve exploiting previously unknown vulnerabilities in IIoT devices or software. Zero-day attacks can be difficult to defend against, as there may not be patches or other security measures available to address the vulnerability.

While IIoT offers numerous benefits, it also poses significant security challenges that require sophisticated solutions such as DL-driven

intelligence-based approaches for detecting and preventing botnets and other cyber threats.

1.3. Intrusion detection system (IDS)

IDS stands for Intrusion Detection System. It is a security technology that monitors network traffic or system events for signs of malicious activity or unauthorized access (Aldweesh, Abdelouahid Derhab, & Ahmed, Emam, 2020). IDS typically works by analyzing network packets, system logs, or other sources of information to identify patterns or anomalies that may indicate an intrusion or security breach as shown in Fig. 3. In the Industrial Internet of Things (IIoT) context, an IDS is a security technology that monitors network traffic for signs of malicious activity or unauthorized access. IIoT systems are often complex and distributed, with multiple devices and sensors connected to a network. This can make them vulnerable to cyber-attacks, such as malware infections, denial-of-service (DoS) attacks, and unauthorized access by malicious actors. An IDS can help to detect these attacks by analyzing network traffic and looking for patterns or anomalies that indicate suspicious activity.

There are two main types of IDS in IIoT: signature-based IDS and anomaly-based IDS (Ioulianou and Vasileios Vasilakis and Ioannis Moscholios & Michael Logothetis, 2018). Signature-based IDS uses a database of known attack signatures to identify malicious traffic. On the other hand, anomaly-based IDS uses machine learning algorithms to learn what is "normal" traffic and flag any traffic that deviates from the norm as potentially malicious. IDS can play a critical role in protecting IIoT systems from cyber-attacks. By detecting and responding to security incidents in real-time, IDS can help to prevent data breaches, system downtime, and other negative consequences of cyber-attacks.

1.4. Motivation

Given the interconnected and interdependent nature of IoT devices, the constant emergence of new threats poses significant challenges. Furthermore, since IoT devices often operate in unattended environments, malicious actors can exploit vulnerabilities to gain unauthorized access. Eavesdropping, specifically through wireless networks connecting IoT devices, further exacerbates the security risks by compromising the confidentiality of privately-owned information. Moreover, due to resource constraints such as limited energy and processing power, incorporating advanced security features into IoT devices is often impractical. Consequently, it becomes imperative to establish an additional layer of protection within IoT networks to safeguard organizations relying on IoT technology from cyber threats.

In recent times, deep learning techniques have gained considerable credibility as an effective means of detecting network attacks in IoT

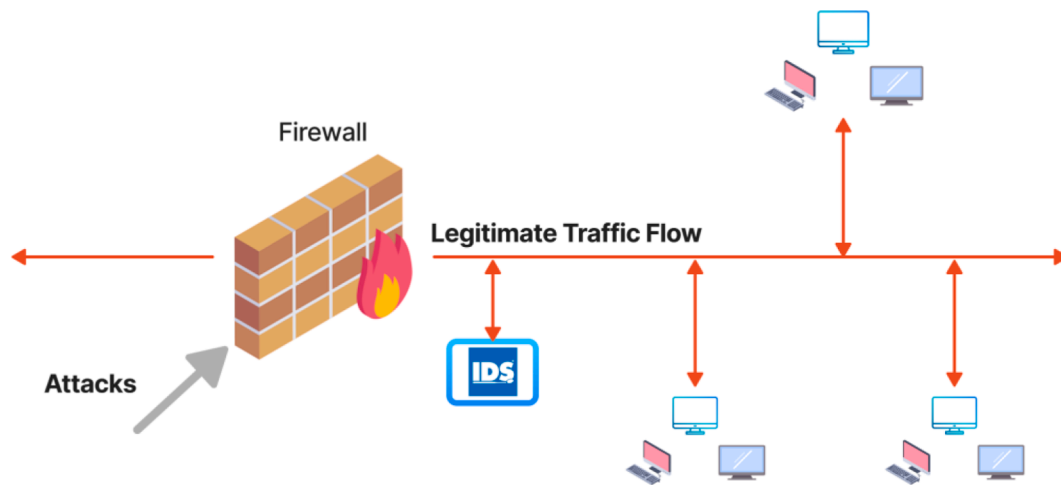


Fig. 3. Working of Intrusion Detection System.

environments. Through extensive training, these systems primarily focus on identifying both anomalous and benign behaviors. By capturing and analyzing data related to IoT devices and network traffic, normal patterns can be established. Consequently, any deviations from these established patterns can be promptly recognized as irregular behaviors. Such approaches have also been evaluated for their efficacy in predicting emerging threats and devising comprehensive security protocols for IoT devices and networks.

The fundamental contribution of the research work is:

- **Designed a robust anomaly detection framework for the IIoT:** The research paper aims to develop a robust anomaly detection framework specifically tailored for the IIoT context. This framework facilitates the improved detection of malicious data generated by compromised and attacked heterogeneous IIoT devices.
- **Proposed an efficient feature extraction model for IIoT-based anomalies:** The paper puts forth an efficient model for extracting robust features from IIoT data while eliminating unnecessary features. This approach enhances the performance of anomaly detection in the IIoT domain.
- **Proposed of an adaptive based CNN-GRU model for IIoT network-based attack detection:** The paper introduces an adaptive hybrid model that integrates Convolutional Neural Networks (CNN) and Gated Recurrent Units (GRU). This model is specifically designed to detect network-based attacks in the Industrial Internet of Things (IIoT) environment.
- **Performance evaluation using a multi-class classification approach:** The proposed model is evaluated and tested using the "Detection of IIoT botnet attacks N_BaIoT" security dataset. The evaluation employs a multi-class classification approach, effectively distinguishing benign traffic from nine different attacks initiated by IIoT botnets.
- **Comparison with state-of-the-art techniques:** The proposed methodology is compared against existing techniques based on various parameters. By conducting this comparative analysis, the paper establishes the effectiveness and superiority of the proposed model in detecting IIoT botnet attacks.

1.5. Structure organization

The research paper is structured into the following sections: [Section 2](#) presents an extensive literature review encompassing existing techniques employed in the detection of botnet attacks. This review aims to identify the prevailing challenges commonly encountered by these methods. In [Section 3](#), a detailed description is provided regarding the

techniques utilized for data acquisition and preprocessing in the study. [Section 4](#) outlines the proposed model, which is primarily grounded in deep learning methodologies. [Section 5](#) elucidates the experimental design and result analysis applied to evaluate the proposed model, along with a comparative assessment against state-of-the-art approaches currently in use. Lastly, [Section 6](#) offers conclusive remarks regarding the findings of the study and highlights potential avenues for future research exploration.

2. Literature Review

In recent years, there has been a growing interest in utilizing Artificial Intelligence (AI) techniques, particularly machine learning (ML) and deep learning (DL), for the development of Network Intrusion Detection Systems (NIDS). While both ML and DL algorithms have been extensively explored, there has been a noticeable shift towards DL due to the advancements in Graphical Processing Units (GPUs) technology. This technological progress has significantly accelerated the computation capabilities required by DL algorithms. Consequently, researchers have increasingly adopted DL algorithms to propose more accurate security solutions for Internet of Things (IoT) networks, which often deal with large volumes of raw data. DL algorithms have proven to be highly effective in analyzing complex patterns by leveraging their deep architecture ([Balan K, Abdulrazak, Khan, Julaihi, Tarmizi, Pillay, & Sallehudin, 2018](#)). This characteristic enables DL models to effectively classify network traffic as either benign or anomalous. This is particularly valuable for intrusion detection in IoT networks, where the diversity and complexity of network traffic patterns pose significant challenges.

However, Deep Learning (DL) encounters several challenges that need to be addressed. One prominent issue arises from the proliferation of network-connected edge devices, leading to an enormous volume of generated data ([Mohanta, Debasish Jena, Utkalika Satapathy, & Srikantha Patnaik, 2020](#)). This data abundance presents computational complexity and high expenses ([Zikria, Muhammad Khalil Afzal, Sung Won Kim, Andrea Marin, & Mohsen Guizani, 2020](#)), demanding substantial performance capabilities and power consumption ([Tang, Jie and Sun, Dawei and Liu, Shaoshan and Gaudiot, & Jean-Luc, 2017](#)). To efficiently utilize the Internet of Things (IoT) and overcome these obstacles, it is paramount to tackle them at their roots. A promising approach involves employing artificial intelligence and big data solutions, focusing on network security itself. This can be achieved by analyzing different deep learning (DL) models, such as recurrent neural networks (RNNs), deep neural networks (DNNs), restricted Boltzmann machines (RBMs), deep belief networks (DBNs), convolutional neural

networks (CNNs), deep Boltzmann machines (DBMs), and deep autoencoder (Ferrag, Leandros Maglaras, Sotiris Moschoyiannis, & Helge Janicke, 2020). Evaluating and improving the feasibility and applicability of these models will help to address the aforementioned challenges and optimize the use of IoT resources.

The security and privacy issues cover a diverse range of topics within the fields of control systems, optimization algorithms, and electronics. This diverse collection of state of art work presents innovative solutions to various engineering challenges. From optimizing a 6-DOF AUV body (Basil, Marhoon, Gokulakrishnan, & Buddhi, 2022) and enhancing an electro-hydraulic system to designing a reconfigurable microstrip patch antenna (Marhoon, Qasem, Basil, & Ibrahim, 2022) and improving an AVR control system (Ibrahim, Basil, & Mahdi, 2021), these works showcase the power of optimization algorithms in achieving significant performance improvements. Techniques like JOA (Basil, Marhoon, & Ibrahim, 2023), BHO (Raheem, & Noorulden Basil, 2023), and ALO (Marhoon, Ibrahim, & Basil, 2021) are employed to address issues like power fluctuations, trajectory control, and stability, demonstrating their effectiveness across diverse applications in robotics, control systems, renewable energy, and aerospace (Mohamadwasel, 2020; Mohammed, Basil, Bayat, & Mohammed, 2020). This research paves the way for further advancements in optimization techniques and their application to enhance the performance and efficiency of various engineering systems. Collectively, these state of art work contributes valuable insights to their respective fields, showcasing innovative approaches and techniques in control systems, optimization algorithms, and electronic design.

Another critical aspect of privacy and security in energy storage and sustainability. The importance of sensing for ensuring the safety (Yi, Chen, Yin, Wang, & Wang, 2023) and longevity of new energy storage devices, the potential of Nano generators in acoustic applications (Yu, Shang, Zheng, & Wang, 2023), and the use of electrochemical impedance spectroscopy (Liu, Wang, Li, & Wang, 2023) and machine learning techniques for estimating the remaining useful life of batteries and super capacitors (Ma, Yin, & Wang, 2023). These advancements contribute to the development of more reliable, efficient, and sustainable energy systems.

In this research manuscript, we present a comprehensive systematic literature review of existing intrusion detection models that employ various techniques. We provide an overview of the research landscape in this domain, highlighting the utilization of ML and DL algorithms. Table 1 summarizes the key findings from the literature review, presenting a comparative analysis of different intrusion detection models and Table 2 represents the publicly available dataset. By employing DL algorithms, researchers aim to enhance the accuracy and reliability of intrusion detection in IoT networks. These models leverage the ability of DL to extract intricate features from raw network data, enabling them to detect subtle anomalies and classify network traffic more effectively. The adoption of DL algorithms holds promise for improving the security and resilience of IoT networks against evolving cyber threat.

In the currently accessible literature, researchers have proposed a range of ways for detecting assaults and irregularities in network systems. It is worth noting that some scholars have focused on employing centrality metrics to address this issue. To accomplish reliable identification, these researchers used a combination of machine learning techniques, feature extraction classification methodologies, and hierarchical clustering. Each study delivers a distinct contribution that, when combined, provides vital insights into network system attack detection. We attempt to solve these concerns through our research contributions after reviewing the various approaches currently in use to identify common challenges. The existing body of literature from Table 1 reveals several noteworthy challenges and limitations in the field of anomaly and attack detection in network systems. These include:

Insufficient Scalability Analysis: Regrettably, many researchers have neglected to evaluate the scalability of their proposed

approaches, which is a critical concern when considering real-world deployment. Consequently, the current methods may lack the necessary scalability to effectively handle the demands of large-scale IoT networks.

Lack of Discourse on Hyper Parameter Tuning: Numerous researchers have omitted discussions on the tuning of hyper parameters for their machine learning and deep learning algorithms. This crucial stage in the ML and DL pipeline significantly influences model performance. Failing to disclose how the best combination of hyper parameters was established hampers reproducibility and hinders the optimization of models.

Inadequate Development of integration of deep learning Models: The development of hybrid models, which integrate multiple machine learning algorithms or other approaches, remains limited in the existing literature. Exploring the benefits of hybrid models for IoT intrusion detection presents a promising avenue for future research, as they have the potential to enhance accuracy and performance.

Limited Comparison with State-of-the-Art Approaches: Most studies have neglected to compare their proposed models with other state-of-the-art ML and DL methods for intrusion detection. This lack of comparison hampers the ability to determine the effectiveness of existing models in relation to alternative approaches, thus impeding progress in the field.

Restricted Evaluation Metrics: The majority of researchers rely on a limited set of evaluation metrics to assess the efficacy of their models. Employing comprehensive assessment measures, such as precision, recall, F1-score, and confusion matrix, commonly found in the literature, would offer a more holistic evaluation of the existing models and facilitate better comparisons.

Limited Results Analysis and Experimentation: Many researchers provide scant analysis of their results and fail to delve into the factors influencing the performance of their models or contributing to their success or failure. A more in-depth exploration and analysis of the results would yield valuable insights into the efficacy of the proposed approaches and aid in identifying areas for improvement.

Moreover, it is worth noting that a majority of research efforts focus predominantly on a limited classification of IoT attack. To ascertain the effectiveness of existing methodologies in identifying other types of IoT attacks, it is crucial to subject them to rigorous testing against various attack scenarios. To overcome these challenges, our research endeavors to develop hybrid deep learning approaches capable of accurately detecting diverse types of anomalies and attacks within the IoT environment. By integrating deep learning techniques, we aim to enhance the dependability and reliability of intrusion detection systems, thus enabling them to effectively address the evolving threat landscape of IoT networks.

3. Proposed methodology

This section introduces our proposed Intrusion Detection System (IDS) model, which is based on deep learning techniques. We begin by presenting the problem statement, followed by an overview of the model we used. Subsequently, we provide a detailed explanation of the structure and algorithm of our proposed model, along with a description of how we adapted the model for the purpose of detecting intrusions also depicted in Fig. 4 in modular representation.

3.1. Problem statement

Assuming the availability of a training dataset $D = \{(X_1, Y_1), \dots, (X_n, Y_n)\}$, where X_i denotes a vector of real or discrete features expressed as $(X_{i1}, X_{i2}, X_{i3}, X_{i4}, \dots, X_{im})$ and X_{ij} represents the j th feature of vector X_i . In contrast, the target variable Y_i has one type i.e. multiclass classification consisting of different classes (1—

Table 1

A Literature Summary of existing work on IDS based on Artificial Intelligence Techniques.

Year and Reference	Purpose	Methodology used	Dataset used	Result	Advantages	Limitations
2023, (Abusitta A et al.)	To design a robust efficient model to extract IoT-based features and improve the detection of malicious attacks.	IoT-based DNN model	DS20S trace dataset	Acc = 94 %	Utilizes a denoising auto encoder in deep neural networks, allowing for robust feature extraction in noisy and heterogeneous IoT environments	Limited or biased training data may lead to suboptimal performance or generalization issues
2023, (Oseni A et al.)	To design a framework for AI-based ID in IoT and IoV environment.	Shapley additive explanation(S HAP) based mechanism	TON-IoT	Acc = 99.15 % for binary classification	Adaptive to known threats	Proposed framework is computationally intensive and expensive to run
2023, (Mehedi et al., 2022)	DTL-based Resnet model is designed for Effective IDS detection on Various cyber threat	P-Resnet model with transfer learning	TON-IoT new generation dataset	Acc = 87 %	Improved performance and dependability in IoT intrusion detection	Needed to optimize hyper parameters and implement privacy-preserving measures
2023, (Bhandari G et al.,)	To design an AI-based framework for IoT cyber security	Deep neural network (DNN)	IoT23, Edge_IIoT dataset	Acc = 93 %	Effective multi-level attack classification and malware detection	Robustness against sophisticated attacks
2023, (Khan F et al.,)	To design a scalable, efficient Deep transfer learning-based cyber-attack detection mechanism	PRU + DT with SCADA-based IIoT model	SCADA-based power system dataset	Acc = 98.89 % For binary classification,95.56 % for multiclass classification	Effective utilization of Enhanced deep and ensemble learning techniques	Less type of attacks captured and classification
2023, (Awotunde JB et al.,)	To develop a multi-level intrusion detection system for IoT	Fuzzy based system and random forest	NSL-KDD	Acc = 99.46 %	Selecting a set of unique Feature for classification	Performance of fuzzy logic is dependent on quality of the knowledge embedded in the system.
2023, (Wang S et al.,)	To develop an intelligent approach for intrusion detection in IoT	Res-tranBiLstm	NSL-KDD, CIC-IDS	Acc = 99.15 %	Considered both spatial and temporal features.	Instability of model for real time analysis.
2023, (Jothi B et al.,)	To develop a novel DL-based intrusion detection system for IoT network	WILS-TRS	OMENT-python APT	Acc = 99 %	Adaptive to trust-based countermeasures to safeguard network.	Prediction time is more as compared to SOTA techniques.
2023, (Heidari A et al.,)	To develop secure intrusion detection platform for Internet of Drone	RBFNNs model using blockchain	UNSW-NB-15, NSL-KDD, CICDDOS 2019, CICIDS 2017, AWID	Acc = 99.11 %	Effective utilization of zero knowledge proof of protocol.	Limited attack classification.
2023, (Khanday SA et al.,)	To develop IDS model for IoT network to mitigate DDoS attack	SVM, LR, Navie bayes, ANN, LSTM	BoT-IoT, To?N-IoT	Acc = 98 %	Specifically designed for lightweight IoT network.	Limited to binary classification only.
2022, (Sharma M et al.,)	To design a model for leveraging anomaly detection using transfer learning in an IIoT environment	Deep progressive neural network (DPNN)	BOT-IoT	Acc = 94 %	Utilizes transfer -learning to leverage information	Low misclassification percentage
2022, (Hasan T et al.,)	To design an efficient, scalable, flexible AI-enabled IDS model for IoT	LSTM-DNN model	N-BaloT dataset	Acc = 99.94 %	Adaptability of the hybrid DL algorithm	Limited attack classification
2021, (Makkar A et al.,)	To design an efficient IoT-based spam detection mechanism	Generalized linear model with stepwise feature selection	REFIT project smart home dataset	Acc = 91.8 %	refined measure to assess the trustworthiness and reliability of IoT devices	Require adaptation and customization for different IoT deployment
2021, (Lv et al.,)	To design an improved auto- encoder for IDS to improve the security performance of IoT	SDAE (stacked auto coding)	NSL- KDD data based on KDD-CUP99 dataset	Acc = 98.12 %	Reduced detection -load	Increased training cost with the number of network layers
2021, (Kumar P et al.,)	To design a privacy-preserving and secure framework for smart cities driven by IoT	Gradient boosting anomaly detection (GBAD) and Blockchain integrated IPFS	TON-IoT, BOT-IoT	Acc = 97.1 %	Integration of blockchain and machine learning forprivacy -protection	Robustness against advanced attacks
2021, (Li B et al.,)	To design a novel based IDS for Industrial CPS on the federated learning framework	CNN + GRU using federated learning (DeeFed)	Real-time data source of a gas pipelining system	Acc = 99.20 %	Privacy-preserving collective intrusion Detection with effective cyber threat detection	Requires cross-domain federated approach
2020, (Arachchige PC et al.,)	Privacy-preserving trustworthy ML framework to address privacy and trust issues	PriModChain (CNN-based framework)	MNIST dataset	Acc=<85 %	The framework excels in privacy, security, reliability, safety, and resilience, ensuring trustworthiness	overhead due to the integration of multiple technologies, such as smart contracts, blockchain, FedML, DP, and IPFS

(continued on next page)

Table 1 (continued)

Year and Reference	Purpose	Methodology used	Dataset used	Result	Advantages	Limitations
2019, (Qiu C et al.,)	in IIoT using federated learning To design a blockchain-based protocol and DQL Approach for optimal analysis	Quelling deep Q-learning approach	–	LR=1e ⁻⁵ due to its better learning stability	Improved synchronization, efficiency, and resource allocation in SDIIoT	Challenge in measuring trust features of nodes and controllers

Table 2

Data analysis of publicly available datasets.

Dataset name and year	IoT enabled	No. of IoT devices connected	Types of attacks	Total features	Description
Edge-IIoT, 2022, (Ferrag MA et al)	YES	10	5	14	A curated collection of data from Industrial Internet of Things (IIoT) devices deployed at the network edge, the Edge IIoT dataset supports research and analysis in the fields of edge computing and industrial automation.
IoT-23, 2020, (Garcia S et al)	YES	3	11	23	The Internet of Things (IoT)-23 dataset is a large collection of network traffic statistics produced by various IoT devices. It includes 23 various IoT device types, such as wearable technology, industrial sensors, and smart home products.
N-BaIoT, 2020, (Meidan Y et al)	YES	9	10	115	The N-BaIoT dataset, which stands for "Network-Based Detection of IoT Botnet Attacks," is a comprehensive and in-depth collection of network traffic data that focuses on the identification and analysis of botnet and malware assaults against IoT devices. The dataset includes numerous IoT device kinds, attack methods, and malware versions, as well as many characteristics of IoT botnet attacks. By simulating actual attack scenarios, it offers a wide range of network traffic information produced by infected IoT devices.
BoT-IoT, 2018, (Koroniotis N et al)	YES	1	6	45	The collection contains a variety of network traffic captures that were collected from actual IoT devices that were compromised by various botnet malware strains. It captures interactions between a variety of IoT device types and command and control (C&C) servers, including cameras, routers, and smart home appliances.
CIC-DDoS2019, (Sharafaldin I et al)	Network based	–	12	80	A comprehensive collection of network traffic statistics primarily targeted against Distributed Denial-of-Service (DDoS) attacks is the CIC-DDoS, 2019 dataset. It was put up for research and analysis purposes and offers insightful information about the traits and tendencies of DDoS attacks. The dataset contains information on a variety of DDoS attacks, including HTTP, TCP, and UDP floods, among others. It provides a comprehensive view of the attack situation by including network traffic captures from the viewpoints of the attacker and the victim.
UNSW_NB15, 2015, (Moustafa N)	Network based	–	9	49	This dataset is a vast collection of network traffic data that was recorded in a supervised lab setting. It was created especially for NIDS (network intrusion detection system) evaluation and study. The dataset includes a variety of network traffic scenarios that simulate several attack types, such as DoS (Denial of Service), probing, and infiltration. These scenarios include both legitimate and malicious operations. To offer a variety of network behaviors, it has nine separate attack categories and numerous attack variants.
NSL-KDD, 2009, (Tavallaei M et al)	Network based	–	4	43	The dataset comprises of network traffic data that illustrates various attack types and standard network operations. It has attributes like statistics unique to a given protocol, connection length, service types, and more. DoS (Denial of Service), Probe, R2L (Remote to Local), and U2R (User to Root) are the four main classifications into which the attacks fall.
KDD-CUP-99, 1999, (Stolfo SJ et al)	Network based	–	4	41	A total of 4,898,431 network connections are included in the dataset, together with information about their source and destination IP addresses, port numbers, types of protocols, and lengths of connections. Denial-of-service (DoS), remote-to-local (R2L), user-to-root (U2R), and probing attacks are only a few of the many attack types it contains.

N). Given this dataset D consisting of Z examples, the objective is to develop a learning algorithm that produces a classifier output as 'T', which is a hypothesis represented by a true function $f(X_i) = Y_i$. This function predicts new values of Y_i for any given input X_i .

3.2. Data pre-processing

3.2.1. Data augmentation

Data augmentation is a technique used in machine learning and data analysis to increase the size and diversity of a dataset by applying various transformations to the existing data. We employed a data augmentation technique to enhance the diversity and size of our dataset, thereby improving the robustness and generalization capability of our machine-learning models. Specifically, we defined a function, namely `augment_data`, to systematically apply transformations to each row of the dataset.

For instance, we introduced randomness into the numerical column labeled 'MI_dir_L5_weight' by adding a randomly generated number from a normal distribution with a mean of zero and a standard deviation of one. This process injected subtle variations into the original values, augmenting the dataset with more diverse instances.

```
row['MI_dir_L5_weight'] = row['MI_dir_L5_weight'] + np.random.normal(0, 1).
```

```
# For example, randomly shuffle the characters in a text column.
# row['type'] = ''.join(np.random.permutation(list(row['type'])))
return row.
```

3.2.2. Data shuffling

Data shuffling is the process of randomly changing the order of data instances in a dataset to introduce randomness and reduce biases. To enhance randomness and reduce biases, row shuffling was performed on the dataframe 'data'. This process involved randomly permuting the row

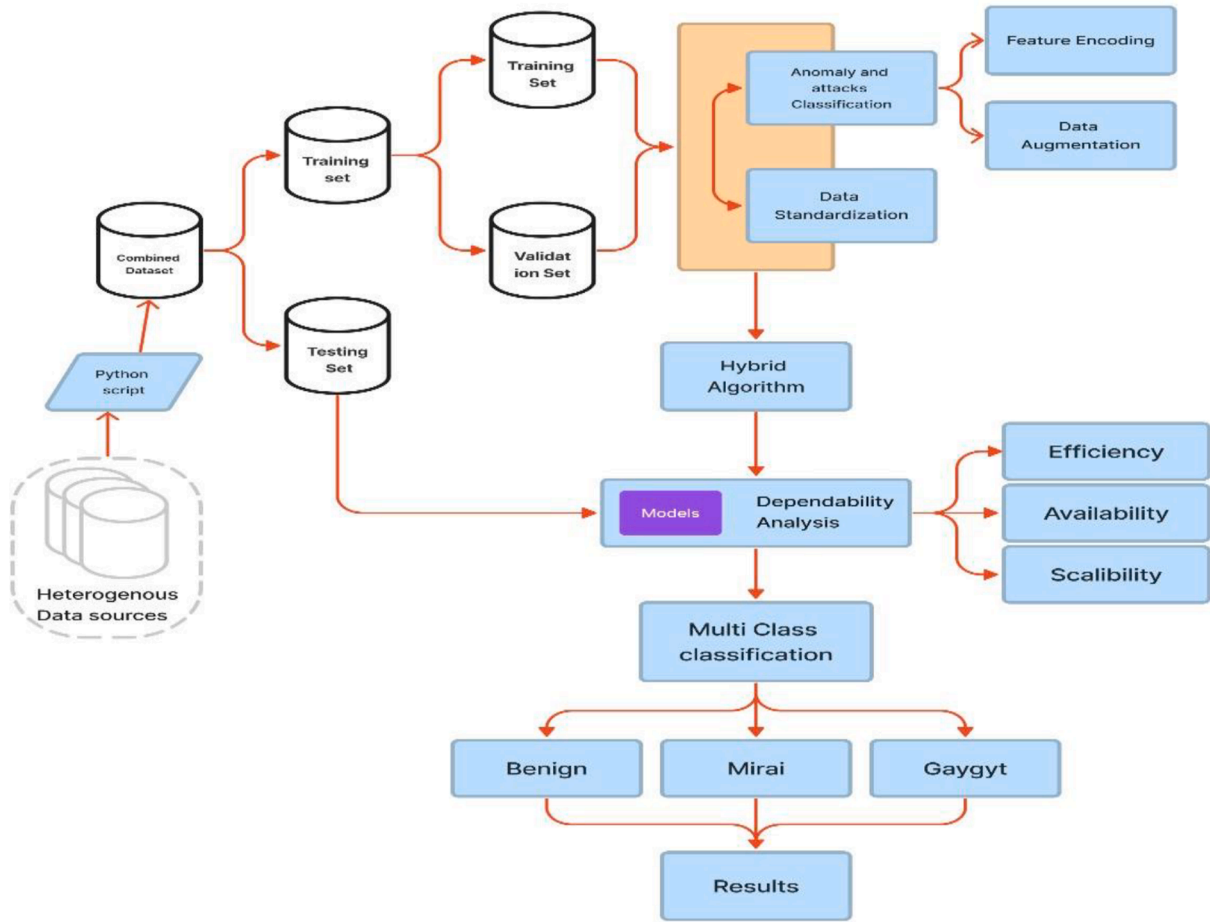


Fig. 4. Schematic Overview of the Modular Architecture in the Proposed Model.

indices using 'np.random.permutation'. The 'take' method was then used to rearrange the rows according to the shuffled indices. This randomized ordering provides a more unbiased representation of the data, which is crucial for reliable research conclusions.

3.2.3. Feature encoding

Feature encoding refers to the process of converting categorical labels into a binary representation using dummy encoding. The encoded labels are stored separately in the 'labels' dataframe, and the original "type" column is dropped from the 'data' dataframe, ensuring separation between the labels and the training dataset. To incorporate categorical labels into the research analysis, dummy encoding was applied. The 'pd.get_dummies' function transformed the categorical labels into binary columns with a unique identifier for each category. The resulting dataframe, 'labels', contained the dummy-encoded columns. The original "type" column was dropped from the 'data' dataframe to separate the encoded labels from the features. This encoding technique allowed for the inclusion of categorical information as meaningful features, enhancing the research analysis comprehensiveness and accuracy.

3.2.4. Data standardization

Data standardization scales numerical values to have zero mean and unit standard deviation. Numerical columns in the 'data' dataframe were standardized to ensure comparability and reduce the impact of different scales. The 'standardize' function was used to transform each column by subtracting the mean and dividing by the standard deviation. The standardized values were stored in a copy of the dataframe named 'data_st'. This process improved the reliability and interpretability of the data for subsequent analysis. The equation for data standardization, also

known as z-score normalization, is:

$$Z = (X - \mu) / \sigma \quad (1)$$

where: Z represents the standardized value, X is the original value, μ is the mean of the data, and σ is the standard deviation of the data.

By applying the z-score formula to each numerical column, the standardization process transforms the values to have a mean of 0 and a standard deviation of 1. This ensures that the numerical columns are on a comparable scale, facilitating fair comparisons and avoiding the influence of varying magnitudes in subsequent analyses.

Algorithm 1 represents preprocesses a raw dataset by adding a new column to indicate attack types, removing similar features, augmenting data, shuffling, encoding labels, standardizing, and splitting the dataset for further analysis.

Algorithm 1: Data preprocessing

```

1. raw dataset = Read dataset(path)
2. procedure = Adding new column "type" which depicts the type of the attack
3. concatenated dataset into single dataset with label
{
  data = pd.concat()
}
4. printf (data N*M) = 926157*116
5. checking features similarities
6. dropping similar feature column
{
  data = data.drop()
}
7. data augmentation -> add noise to the column
{
  aug_data = df.apply()

```

(continued on next page)

(continued)

Algorithm 1: Data preprocessing

```

aug_data to CSV()
}
8. printf (data N*M) = 926157*76
9. shuffle (data)-> Random permutation
10. encode (labels)
11. dataset = labels-> dummy encoding
12. for each column and row
13. data = type, prefix = type
14. standardize (dataset)
{
column_mean/std;
}
15. end procedure
16. procedure (split dataset)
{
80:20 ratio
}
17. end procedure

```

3.3. CNN

Convolutional Neural Networks (CNNs) have demonstrated outstanding performance in various fields such as computer vision, speech recognition, and natural language processing. For Intrusion Detection Systems (IDS), CNNs extract and classify features, surpassing traditional methods. The name “convolutional” stems from the matrix-based linear operation called convolution (Jo W, Kim S, Lee C, & Shon, 2020). CNNs, or Covnets, enable parameter sharing and comprise layers like input, convolution, pooling, fully connected, and output. The convolution layer convolves input data with filters to detect patterns, while pooling reduces spatial dimensionality (Yao R, Wang N, Liu Z, Chen P, & Sheng X, 2021), capturing essential information as shown in equation 2–5. Fully connected layers link extracted features to the output layer for classification. CNN models contain multiple hidden layers, extracting information from images represented as pixel matrices. Fig. 5 illustrates the generic CNN model structure, depicting the layer arrangement.

$$y_j = b_j + \sum_{c=0}^{n_c-1} \sum_{k=-p}^p x_{c,j-k} \mathcal{W}_{c,k} \text{convolution} \quad (2)$$

$$\frac{\partial \mathcal{L}}{\partial x_{c,j}} = \sum_{k=-p}^p \frac{\partial \mathcal{L}}{\partial y_{i+k}} \mathcal{W}_{c,k} \text{inputgradient} \quad (3)$$

$$\frac{\partial \mathcal{L}}{\partial w_{c,k}} = \sum_{j=0}^{m-1} \frac{\partial \mathcal{L}}{\partial y_j} x_{c,j-k} \text{parametergradient} \quad (4)$$

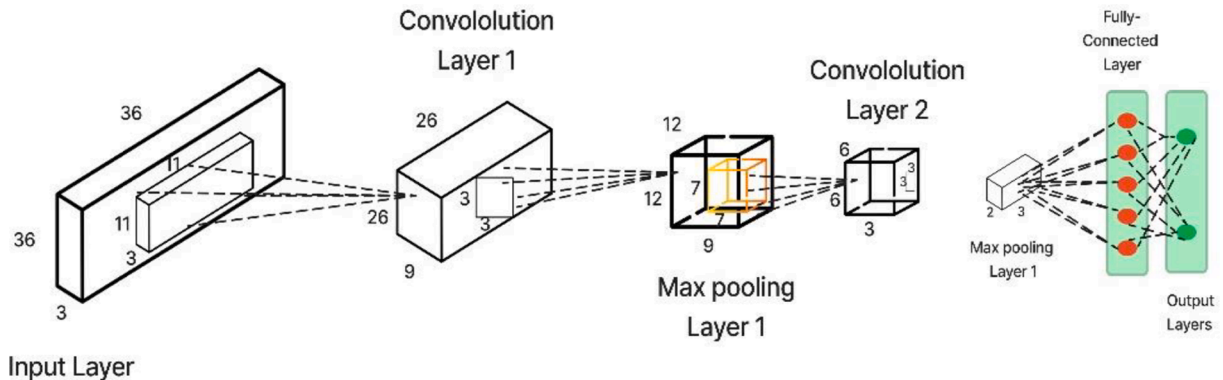


Fig. 5. Generic representation of convolution layer and max pooling layer operation in CNN.

$$\frac{\partial \mathcal{L}}{\partial b_j} = \frac{\partial \mathcal{L}}{\partial y_j} \text{biasgradient} \quad (5)$$

3.4. GRU

Gated Recurrent Units (GRUs) have demonstrated exceptional performance in various domains such as natural language processing, speech recognition, and sequential data analysis. In the context of Intrusion Detection Systems (IDS), GRUs are utilized for their ability to effectively model and understand sequential data. Unlike traditional recurrent neural networks, GRUs employ gating mechanisms to regulate the flow of information (Zhai F, Yang T, Chen H, He B, & Li S, 2023). They consist of input, forget, and output gates, which enable them to selectively update and utilize relevant information as shown in Fig. 6. GRUs can capture long-term dependencies while mitigating the vanishing gradient problem. These properties make GRUs well-suited for processing and classifying sequential data in IDS applications.

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \quad (6)$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \quad (7)$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t]) \quad (8)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t \quad (9)$$

3.5. Proposed model

The proposed model is a deep learning architecture for classification tasks. It is a sequential model that consists of convolutional (Conv1D) and gated recurrent unit (GRU) layers, followed by dense layers. Table 3 shows the layers in the proposed model. The model takes input in the form of time-series data, where the input shape is (train_data.shape [1], 1), which represents the number of time-steps and the number of features in each time step, respectively. The first layer of the model is a Conv1D layer with 64 filters, a kernel size of 5, and a stride of 1. The padding parameter is set to “same” to keep the input and output

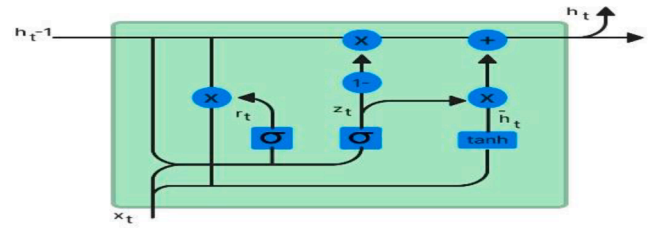


Fig. 6. Generic representation of Gated Recurrent Units (GRU).

Table 3

Layer wise details of the proposed model AttackNet.

Layers	Output Shape	Number of parameters
Conv 1D	(none, 75,64)	384
Conv 1D	(none, 75,32)	10,272
Max pooling layer	(none, 18,32)	0
GRU 1	(none, 18,32)	6336
GRU 2	(none, 18,16)	2400
Flatten	(none, 288)	0
Dense 1	(none, 128)	36,992
Dense 2	(none, 64)	8256
Dropout	(none, 64)	0
Dense 3	(none, 10)	650
Total parameters		65,290

dimensions the same. The Conv1D layer is followed by another Conv1D layer with 32 filters, a kernel size of 5, and a stride of 1. The padding parameter is again set to “same”. The next layer is a MaxPooling1D layer with a pool size of 4. This layer reduces the spatial dimensions of the output volume, which reduces the computational complexity of the model and helps prevent overfitting.

Next, there are two GRU layers, each with 32 and 16 units, respectively. The activation function used in the first GRU layer is “relu”, while the second GRU layer returns a sequence of vectors of dimension 16. The output of the second GRU layer is then flattened using the Flatten () layer. The next layer is a Dense layer with 128 units and an activation function of “relu”. This is followed by another Dense layer with 64 units and an activation function of “relu”. Finally, there is a Dropout layer with a dropout rate of 0.1, which helps prevent overfitting. The model graphical representation of the model’s architecture as shown in Fig. 7.

The output layer of the model is a Dense layer with a softmax activation function. The number of units in this layer is equal to the number of classes in the classification task. The model is compiled and trained using appropriate loss and optimization functions, along with suitable hyperparameters. The evaluation metrics used to evaluate the model are also shown in the Table 4.

First, define the input sequence X as a matrix of size (T, 1), where T is the length of the sequence:

Table 4

Hyperparameter values of the proposed model.

Hyperparameter	Values
Conv 1D filters	64
Conv 1D kernel size	5
Conv 1D strides	1
Conv 1D padding	Same
GRU 1 unit	32
GRU 1 activation function	Relu
GRU 2 unit	16
Dense 1 unit	128
Dense 1 activation function	Relu
Dense 2 unit	64
Dense 2 activation function	Relu
Output of Dense unit	Number of classes
Output dense activation function	Softmax

$$X = [x(1), x(2), \dots, x(T)] \quad (10)$$

The first layer is a 1D convolutional layer with 64 filters, a kernel size of 5, and a stride of 1 and then define the weight matrix $W[1]$ of size (5, 1, 64) and the bias vector $b[1]$ of size (64) for this layer. Then, the output sequence $Z[1]$ can be obtained as:

$$Z[1][i] = f1 \left(\sum_{j=1,2,3,4,5} W[1][j] * X[i+j-3] + b[1] \right) \quad (11)$$

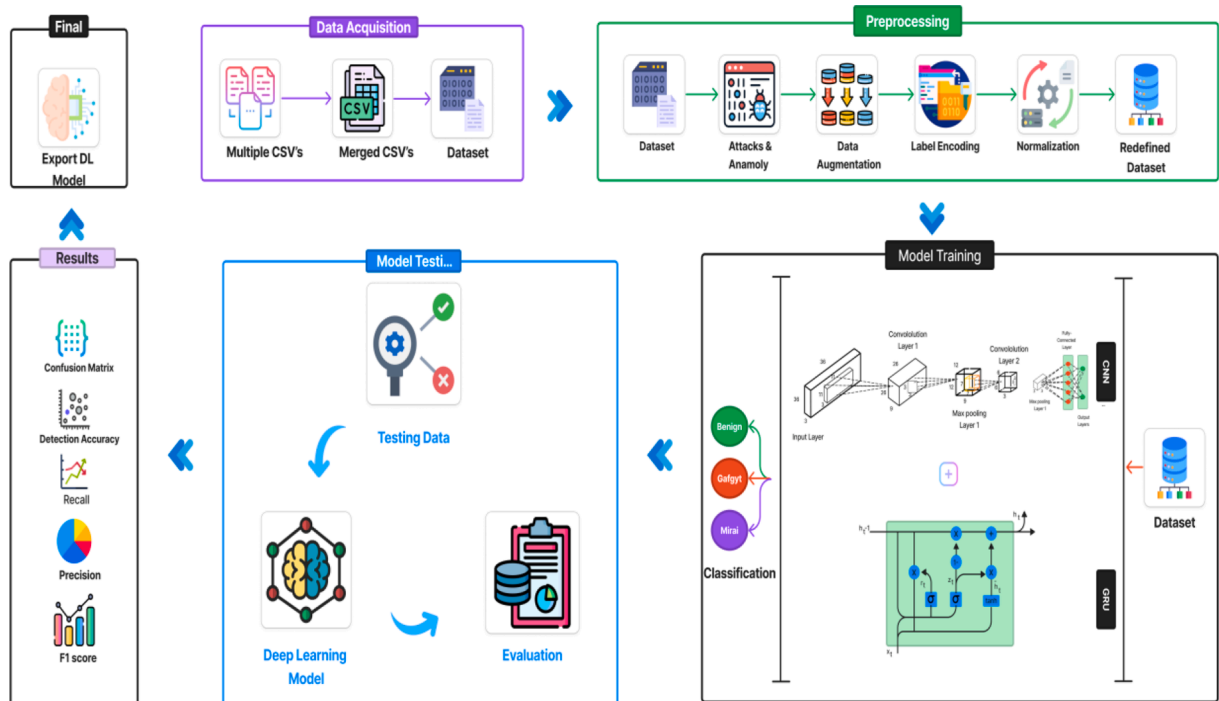
Where f1 is the activation function (e.g. ReLU) and i ranges from 1 to T.

The second layer is another 1D convolutional layer with 32 filters, a kernel size of 5, and a stride of 1. Defining the weight matrix $W[2]$ of size (5, 64, 32) and the bias vector $b[2]$ of size (32) for this layer. Then, the output sequence $Z[2]$ can be obtained as:

$$Z[2][i] = f2 \left(\sum_{j=1,2,3,4,5} W[2][j] * Z[1][i+j-3] + b[2] \right) \quad (12)$$

Where f2 is the activation function (e.g. ReLU) and i ranges from 1 to T.

The third layer is a MaxPooling1D layer with a pool size of 4. This layer reduces the length of the sequence by a factor of 4 which define the

**Fig. 7.** Comprehensive Systematic Representation Elaborating the Architecture and Key Components of the Proposed Model (AttackNet).

output sequence of this layer as $Z[3]$, which has the length $T/4$. Then, the output sequence $Z[3]$ can be obtained as:

$$Z[3][i] = \max(Z[2][4*(i-1)+1], Z[2][4*(i-1)+2], Z[2][4*(i-1)+3], Z[2][4*(i-1)+4]) \quad (13)$$

The fourth layer is a GRU layer with 32 units. Define the weight matrices and bias vectors for this layer as follows:

- $W[3][r]$ Is the weight matrix of size (32, 32) for the reset gate.
- $W[3][z]$ Is the weight matrix of size (32, 32) for the update gate.
- $W[3][h]$ Is the weight matrix of size (32, 32) for the candidate hidden state.
- $U[3][r]$ Is the weight matrix of size ($T/4$, 32) for the reset gate.
- $U[3][z]$ Is the weight matrix of size ($T/4$, 32) for the update gate.
- $U[3][h]$ Is the weight matrix of size ($T/4$, 32) for the candidate hidden state.
- $b[3][r]$ Is the bias vector of size (32) for the reset gate.
- $b[3][z]$ Is the bias vector of size (32) for the update gate.
- $b[3][h]$ Is the bias vector of size (32) for the candidate hidden state.

Where, $h(t)$ be the hidden state at time t , and $r(t)$ and $z(t)$ be the reset and update gates, respectively. Then, the equations for the hidden state, reset gate, and update gate can be written as follows:

$$r(t) = \sigma(W[3][r]*h(t-1) + U[3][r]*Z[3][t] + b[3][r]) \quad (14)$$

$$z(t) = \sigma(W[3][z]*h(t-1) + U[3][z]*Z[3][t] + b[3][z]) \quad (15)$$

$$h'(t) = \tanh(W[3][h]*(r(t)*h(t-1)) + U[3][h]*Z[3][t] + b[3][h]) \quad (16)$$

$$h(t) = (1 - z(t))*h(t-1) + z(t)*h'(t) \quad (17)$$

Where σ is the sigmoid function.

The fifth layer is another GRU layer with 16 units. Define the weight matrices and bias vectors for this layer as follows:

- $W[4][r]$ Is the weight matrix of size (16, 16) for the reset gate.
- $W[4][z]$ Is the weight matrix of size (16, 16) for the update gate.
- $W[4][h]$ Is the weight matrix of size (16, 16) for the candidate hidden state.
- $U[4][r]$ Is the weight matrix of size ($T/4$, 16) for the reset gate.
- $U[4][z]$ Is the weight matrix of size ($T/4$, 16) for the update gate.
- $U[4][h]$ Is the weight matrix of size ($T/4$, 16) for the candidate hidden state.
- $b[4][r]$ Is the bias vector of size (16) for the reset gate.
- $b[4][z]$ Is the bias vector of size (16) for the update gate.
- $b[4][h]$ Is the bias vector of size (16) for the candidate hidden state.

Where, $h'(t)$ be the hidden state at time t for this layer. Then, the equations for the hidden state, reset gate, and update gate can be written as follows:

$$r(t) = \sigma(W[4][r]*h'(t-1) + U[4][r]*h(T/4) + b[4][r]) \quad (18)$$

$$z(t) = \sigma(W[4][z]*h'(t-1) + U[4][z]*h(T/4) + b[4][z]) \quad (19)$$

$$h''(t) = \tanh(W[4][h]*(r(t)*h'(t-1)) + U[4][h]*h(T/4) + b[4][h]) \quad (20)$$

$$h'(t) = (1 - z(t))*h'(t-1) + z(t)*h''(t) \quad (21)$$

The sixth layer is a Flatten layer, which reshapes the output of the previous layer into a 1D vector. The seventh layer is a fully connected layer with 128 neurons and ReLU activation. This layer applies the rectified linear unit (ReLU) activation function to the output of the previous layer. The eighth layer is another fully connected layer with 64 neurons and ReLU activation. This layer also applies the ReLU activation function to the output of the previous layer. The ninth layer is a Dropout layer with a rate of 0.1. This layer randomly sets 10 % of the input units

to 0 at each update during training time, which helps to prevent overfitting. Overfitting occurs when a model learns the training data too well and is unable to generalize to new data. The final layer is another fully connected layer with softmax activation. This layer outputs a probability distribution over the possible classes. For example, if there are 10 classes, the output of this layer will be a 10-dimensional vector, where each dimension represents the probability that the input belongs to a particular class. The structure of proposed architecture AttackNet is shown in Fig. 8.

In the context of multiclass classification tasks on large CSV datasets, the preference for Gated Recurrent Units (GRUs) over other recurrent neural networks such as Long Short-Term Memory (LSTM) and Transformers can be attributed to three key factors. Firstly, GRUs exhibit efficiency and simplicity by having fewer parameters and demanding less computational power than LSTMs, resulting in faster training and deployment, a crucial advantage for handling extensive datasets. The simplicity of GRU's architecture with only two gates facilitates easier hyperparameter tuning and potentially reduces the risk of overfitting in complex tasks associated with large CSV datasets. Secondly, in terms of long-term dependency handling, GRUs prove effective for datasets with shorter sequences, mitigating concerns related to vanishing or exploding gradients that can affect the performance of LSTMs. Lastly, the absence of an attention mechanism in GRUs distinguishes them from Transformers, making GRUs a more cost-effective choice for tasks with shorter sequences where the additional complexity of Transformers may not be necessary, thus ensuring efficient and effective multiclass classification. The fusion of Convolutional Neural Networks (CNN) and Gated Recurrent Units (GRU) offers a robust solution for multiclass classification in CSV datasets, leveraging their respective strengths in spatial and temporal data analysis. CNN layers effectively extract spatial features, discerning intricate patterns within the feature space of CSV columns. Simultaneously, GRU layers, designed for sequential data, capture temporal dependencies by maintaining a memory of past inputs. The concurrent operation of CNN and GRU facilitates a comprehensive analysis of both spatial and temporal aspects, enabling the model to discern complex patterns and dependencies crucial for accurate multiclass classification. The adaptability of this architecture to varying input shapes ensures versatility across diverse CSV dataset structures, contributing to its efficacy in sophisticated classification tasks. Overall, the model combines the power of convolutional neural networks and recurrent neural networks to extract features from the input sequence and learn temporal dependencies in the data. Convolutional neural networks are good at extracting features from images, while recurrent neural networks are good at learning temporal dependencies. By combining these two types of networks, the model is able to learn both spatial and temporal features from the input sequence.

The algorithm 2 describes the Fine-Tuning procedure performs iterative gradient descent to optimize the parameters of a deep learning model using a given dataset, learning rate, epochs, batch size, hidden layers, and initial parameters. It updates the parameters based on the gradients and selects the optimal parameters based on the validation loss.

Algorithm 2: Hyperparameter tuning(fine_tuning)

```

procedure FineTuning( $d, lr, epoch, b, h, \theta$ )
Require:  $d = d1, 2, \dots, dn$ 
Require:  $h = h1, h2, \dots, hz$ 
Require:  $\theta = [\theta1 - \theta z]$ ,  $\theta1 = \text{IoT\_DA\_Training}(x, lr, e, b, \theta1)$ 
Require:  $O = O1, O2, \dots, Ol \triangleright$  hidden layer's output
while  $e \leq epoch$  do
{
Cost Value =  $1/|D| = L(\theta = \{W, b\}, D)$ 
 $grt =$  determine the gradient with respect to  $\theta$ 
}
while  $\theta_i.grti \in (\theta.grti)$  do
{
 $\theta_i = \theta_i - lr * grti$ 
}

```

(continued on next page)

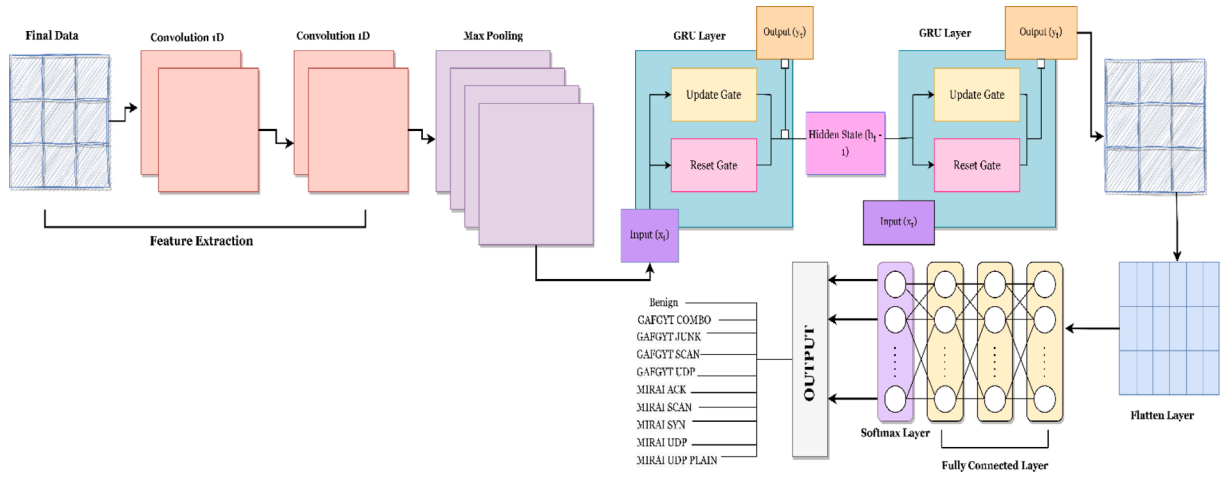


Fig. 8. Architecture of Proposed model AttackNet.

(continued)

Algorithm 2: Hyperparameter tuning(fine_tuning)

```

end while
if
{
Valid Loss < optimal valid Loss then
OptimalEpoch = epoch
optimalParams = θ
optimalvalidLoss = validLoss
}
end if
{
return optimal Parameters
}
end procedure

```

The algorithm 3 involves building a sequential model with Conv1D and GRU layers, followed by Flatten and Dense layers. The model is compiled with specified optimizer, loss function, and metric. It is then trained on the training data, evaluated on the test data, and used to make predictions on new data. The performance of the model is assessed based on accuracy and other metrics.

Algorithm 3: Proposed AttackNet model**Input:** list of attacks, detection model, feature set, timesteps, and training data.**Output:** classification of different botnet attacks.**Function Build CNN GRU Model(train_data, labels):****Define the model architecture**

```

model = Sequential()
model.add(Conv)
layer_num = 1
while {
layer_added == False:
if {
layer_num == 1:
model.add(Conv1D(64, kernel_size = 5, strides = 1, padding='same',
activation='relu'))
layer_num += 1
}
Elif{
layer_num == 2:
model.add(Conv1D(32, kernel_size = 5, strides = 1, padding='same',
activation='relu'))
layer_num += 1
}
else:
layer_added = True
}}
model.add(MaxPooling1D(pool_size = 4))
model.add(GRU)
{
layer_num = 1

```

(continued on next column)

(continued)

Algorithm 3: Proposed AttackNet model

```

while{
layer_added == True:
if{
layer_num == 1:
model.add(GRU(32, activation='relu', return_sequences = True))
}
layer_num += 1
elif {
layer_num == 2:
model.add(GRU(16, return_sequences = True))
}
layer_num += 1
else:
layer_added = False
}}
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.1))
model.add(Dense(num_classes, activation='softmax'))
Compile the model
optimizer = Adam()
loss = 'categorical_crossentropy'
metrics = ['accuracy']
model.compile(optimizer = optimizer, loss = loss, metrics = metrics)
Train the model
epochs = 10
batch_size = 32
validation_data = (val_data, val_labels)
iteration = 1
while iteration <= epochs:
if {
iteration == 1:
model.fit(train_data, labels, epochs = 1, batch_size = batch_size, validation_data =
validation_data)
iteration += 1
}
Else {
model.fit(train_data, labels, epochs = 1, batch_size = batch_size)
iteration += 1
}
return model
Evaluate the model
test_data, test_labels = load_test_data()
loss, accuracy = model.evaluate(test_data, test_labels)
Use the model for predictions
new_data = load_new_data()
predictions = model.predict(new_data)

```

4. Experimental setup and result analysis

4.1. Experimental setup

The study was conducted on a laptop with an Intel Core i5 10th Gen processor, 8 GB RAM, 512 GB ROM, and a Windows 11 operating system. The laptop also had an NVIDIA GTX 1650 GDDR6 4 GB graphics card, which helped to improve performance. The study used various data analysis frameworks, including Pandas, Numpy, Seaborn, Matplotlib, and Scikit-learn. The study carefully considered the laptop's memory limitations of 8 GB RAM.

The study split the data into 80 % training data, 20 % testing data, and 20 % validation data. The proposed model was trained on the training data. The proposed model has two 1D convolutional layers, one max pooling layer, two gated recurrent unit (GRU) layers, and three dense layers. The convolutional layers each have 64 and 2 neurons with kernel size 5. The dense layer has 64 neurons, and the final layer has neurons equal to the number of classes in the dataset. The Relu activation function was used on each dense layer, and the Softmax activation function was used on the final layer. The Adam optimizer and Categorical cross-entropy loss function were used during model compilation for weight update. The study tuned the proposed model using various parameters and hyperparameters to increase the classification accuracy and avoid overfitting. A dropout rate of 0.1 was used, and the number of hidden layers and neurons in each layer were varied. Different learning rates were applied using the Adam optimizer, and it was found that the default learning rate of 0.001 gave better results.

4.2. Dataset Description

In this work, we used the N_BaIoT dataset, which is a publicly available dataset of network traffic data from 9 IoT devices as shown in the Table 5. The dataset includes 115 features and has been used in previous research for botnet detection in IoT/IIoT environments (Meidan Y, Bohadana M, Mathov Y, Mirsky Y, Shabtai A, Breitenbacher D, & Elovici Y, 2018).

The N_BaIoT dataset includes 10 classes of attacks, one of which is BASHLITE. BASHLITE attacks involve scanning for vulnerable devices, sending spam data, UDP flooding, and a combination of sending spam data and opening a connection to a specified IP address & port. Another attack class in the N_BaIoT dataset is Mirai. Mirai attacks involve automatic scanning for vulnerable devices, Ack flooding, Syn flooding, UDP flooding, and UDP flooding with fewer options optimized for higher packets per second. The complete dataset is divided into 10 attack classes and one benign traffic class as shown in the Table 6.

4.3. Performance evaluation metrics

We examined the outcome of the proposed model for different attack types in the dataset. The effectiveness of the proposed model is assessed here using six performance indicators of accuracy, recall, precision, F1 score, Roc, and confusion matrix which are defined as shown in Table 7.

Table 5

IoT devices connected in the dataset.

Device name	Type of device
Danmini	Doorbell
Ennio	Doorbell
Ecobee	Thermostat
Philips B120N/10	Baby monitor
Provision PT-737E	Security Camera
Provision PT-838	Security Camera
SimpleHome XCS7-1002-WHT	Security Camera
SimpleHome XCS7-1003-WHT	Security Camera
Samsung SNH 1011 N	Webcam

Table 6

Dataset Description and Attack Statistics: Target Classes and Instance Counts for Each Attack Type.

Target class and class type	Description of target classes	Attack type	Count
Benign(Class 0)	Unharmful network data	BASHLITE attack	49,548
gafgyt_combo (Class 1)	Combines spam data and connection opening to a specified IP & port	BASHLITE attack	59,718
gafgyt_junk(Class 2)	Sending spam data to device	BASHLITE attack	29,068
gafgyt_scan(Class 3)	Network scan for vulnerable devices	BASHLITE attack	29,849
gafgyt_udp(Class 4)	Flood targeted devices with UDP devices	BASHLITE attack	105,874
mirai_ack(Class 5)	Flood targeted devices with ACK packets	Mirai attack	102,195
mirai_scan(Class 6)	Automatic scan for vulnerable devices	Mirai attack	107,685
mirai_syn(Class 7)	Flood targeted devices with SYN packets	Mirai attack	122,573
mirai_udp(Class 8)	Flood targeted devices with UDP packets	Mirai attack	237,665
mirai_udp plain (Class 9)	Optimized UDP flooding with fewer options for higher packets per second.	Mirai attack	81,982

4.4. Analysis of result

The overall performance of the proposed model was extensively evaluated using various quantitative metrics, including accuracy, precision, recall, receiver operating characteristic (ROC) area under the curve (AUC), F1 score, and the confusion matrix. We also focused on optimizing the model's hyperparameters to enhance its performance. In recent years, deep learning (DL) algorithms have made significant advancements, and their application to intrusion detection and classification problems has been successful. The remarkable progress of hybrid DL approaches has positioned them as a promising solution for reliable and trustworthy network intrusion detection systems (IDS).

Our proposed model, which combines Convolutional Neural Network (CNN) and Gated Recurrent Unit (GRU) layers, demonstrated optimal performance. It achieved a training accuracy of 99.77 % and a testing accuracy of 99.75 %, with a loss of 0.0063 as shown in the Table 8. The precision and recall values were 99.75 % and 99.74 %, respectively. The model employed four hidden layers, including two convolutional layers and two GRU layers. Rectified Linear Unit (ReLU) was used as the activation function in the hidden layers, while softmax served as the output activation function. The categorical cross-entropy loss function, coupled with the Adam optimizer, was utilized.

In the context of Fig. 9 (a) and (b), we evaluate the efficacy of the proposed AttackNet model by elucidating the interplay between training and testing accuracy and training and testing loss. The horizontal axis of both figures delineates the progression of epochs, while the vertical axis corresponds to accuracy and loss, respectively. Notably, a discernible trend emerges wherein minimal epochs coincide with diminished accuracy; conversely, an augmentation in epochs correlates with an enhancement in accuracy. This dynamic indicates the model's iterative learning process, progressively refining its ability to discern patterns within the training dataset. Additionally, Fig. 9 (b) serves as a graphical representation of the model's performance, specifically portraying the loss. Initially, the loss magnitude is considerable with a sparse number of epochs. However, with the advancement of epochs, there is a discernible reduction in loss magnitude. This nuanced depiction encapsulates the model's refinement over time, signifying an improvement in its capacity to minimize errors and enhance overall performance.

After training the model on the N_BaIoT dataset and optimizing the hyper parameters, we achieved impressive performance on the test and validation sets. The model achieved an AUC score of 1.00 on the test set

Table 7
Performance Evaluation Metrics.

Metrics	Definition	Formula
Confusion matrix	Used to evaluate the performance of a classification model by comparing the predicted labels with the actual labels.	
True positive (TP)	Record is successfully detected as malicious	
False positive (FP)	Record is wrongly detected as malicious.	
True Negative (TN)	Record is classified as non-malicious.	
False Negative (FN)	Record is undetected by the system.	
Accuracy	Measure of how well the model predicts the correct labels.	$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$
Precision	Measure of how many of the predicted positive labels are actually positive.	$\text{Precision} = \frac{TP}{TP + FP}$
Recall	Measure of how many of the actual positive labels are correctly predicted.	$\text{Recall} = \frac{TP}{TP + FN}$
F1 score	Harmonic mean of Recall and Precision.	$F1\text{Score} = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$
ROC curve	Graphical representation of the performance of a classification model. TPR is ratio of true positive predictions to the total actual positive labels. FPR is ratio of false positive prediction to the total actual negative labels.	$\text{TPR} = \frac{TP}{TP + FN}, \text{FPR} = \frac{FP}{FP + FN}$
True Negative Rate (TNR)	Model's ability to correctly classify instances of a specific attack type as non-attacks.	$\text{TNR} = \frac{TN}{TN + FP}$
Negative Predictive Value (NPV)	Model's accuracy in predicting non-attacks for a specific attack type.	$\text{NPV} = \frac{TN}{TN + FN}$
False Positive Rate (FPR)	Rate at which instances of other attack types are incorrectly classified as the specific attack type.	$\text{FPR} = \frac{FP}{FP + TN}$
False Negative Rate (FNR)	Rate at which instances of a specific attack type are incorrectly classified as non-attacks or other types.	$\text{FNR} = \frac{FN}{TP + FN}$
False Discovery Rate (FDR)	Rate at which instances are falsely predicted as the specific attack type when they are not.	$\text{FDR} = \frac{FP}{FP + TN}$
False Omission Rate (FOR)	Rate at which instances of a specific attack type are falsely classified as non-attacks or other types	$\text{FOR} = \frac{FN}{FN + TN}$

Table 8
Quantitative analysis of proposed model.

Dataset	Accuracy	Loss	Precision	Recall	AUC
Train set	0.9977	0.0062	0.9978	0.9976	1.0000
Validation set	0.9977	0.0065	0.9978	0.9976	0.9999
Testing set	0.9975	0.0063	0.9975	0.9974	1.0000

and 0.99 on the validation set, as demonstrated in the Fig. 10 (a) and (b).

To evaluate the model's performance on different attack classes, we computed evaluation metrics such as precision, recall, F1 score, and support as shown in the Table 9.. Across all classes, precision, recall, and F1 score consistently exhibited high values. This suggests that the model accurately classified various types of botnet attacks, achieving high accuracy and effectively identifying instances of each attack class.

Precision values of 1 indicate a high level of precision for all attack classes, implying that when the model predicted instances as a specific type of botnet attack, it was highly likely to be correct. Similarly, recall values of 1 indicate that the model demonstrated a substantial ability to

accurately identify and capture a significant majority of instances belonging to each specific attack class. Consequently, the model exhibited a remarkable ability to detect instances of diverse botnet attacks effectively. Moreover, F1 scores approaching 1 indicated a commendable balance between precision and recall for all attack classes. This exemplified the model's heightened accuracy while adeptly capturing instances from each attack class.

To provide a detailed breakdown of the model's predictions for each attack class, we utilized a confusion matrix. The confusion matrix for the train, validation, and test sets enabled a comprehensive analysis of the model's performance as shown in the Fig. 11 (a), (b) and (c). We also assessed additional metrics, including True Negative Rate (TNR), Negative Predictive Value (NPV), False Positive Rate (FPR), False Negative Rate (FNR), False Discovery Rate (FDR), and False Omission Rate (FOR), for each class representing different types of botnet attacks shown in the Table 10.

In evaluating the model's overall performance, these metrics enabled the identification of specific classification issues within classes, facilitating hyper parameter optimization. Fig. 11, 12 and 13 depicting TNR vs. NPV, FPR vs. FNR, and FDR vs. FOR illustrated the trade-offs between correct identification of non-attack instances, avoidance of false alarms, and accurate identification of attack instances, respectively.

The Fig. 12 shows the TNR v/s NPV on different classes. Higher value of TNR indicates a better ability to correct identify non attack instances whereas a higher value of NPV indicates a better ability to correctly identify non attacks instances among the predicted negative. An increased in TNR generally leads to an increase in NPV and vice versa.

The Fig. 13 shows the FPR v/s FNR on different classes. Lower value of FPR indicates a better ability to avoid false alarms for non-attack instances whereas lower value of FNR indicates a better ability to correctly identify attack instances. As FPR increases, FNR tends to decrease and vice versa.

The Fig. 14 shows the FDR v/s FOR on different classes. As FDR increases, FOR tends to decrease and vice versa. When the model makes more incorrect positive prediction (higher FDR), it tends to make fewer incorrect negative predictions (lower FOR), reflecting the trade-off between making incorrect positive and negative predictions.

The class-wise confusion matrix depicted in the Fig. 15 provides valuable insights into the performance of our proposed AttackNet deep learning model. It enables us to analyze the true negative rate, false positive rate, valid positive rate, and false negative rate for each class. Upon analysis, we found that the benign class constituted 5.38 % of the dataset, while the gafgyt_junk and gafgyt_combo classes accounted for only 3.20 % and 2.94 % respectively. Furthermore, the Mirai_ack class exhibited a false positive rate of 11.60 %, indicating a relatively higher misclassification of benign instances as Mirai_ack attacks. Similarly, the Mirai_scan class had a false positive rate of 13.36 %. Overall, we observed that the Mirai_syn attack achieved a remarkably high detection rate, demonstrating the effectiveness of our approach in identifying instances of this specific attack type.

The Fig. 16 shows the TNR value of each classes. It represents the proportion of true negative prediction out of all actual negative instances. The range of TNR is from 0 to 1, with 0 indicating no true negative predictions and 1 indicating perfect performance in identifying negative instances. It gives insight into the model's performance in classifying negative prediction which help to assess the model specificity and its ability to minimize false positive errors. The Fig. 17 shows the FNR values of each attack types. It represents the proportion of false negative predictions out of all actual positive instances. The range of FNR is from 0 to 1, with 0 indicating no false negative predictions and 1 indicating a high rate of false negative prediction. A lower value of FNR indicates a lower rate of missing positive instances, meaning the model is more sensitive in identifying true positive class.

In our study, we have expanded the evaluation of our recursive network's performance by incorporating the Matthews Correlation Coefficient (MCC) as an additional performance metric. The inclusion of

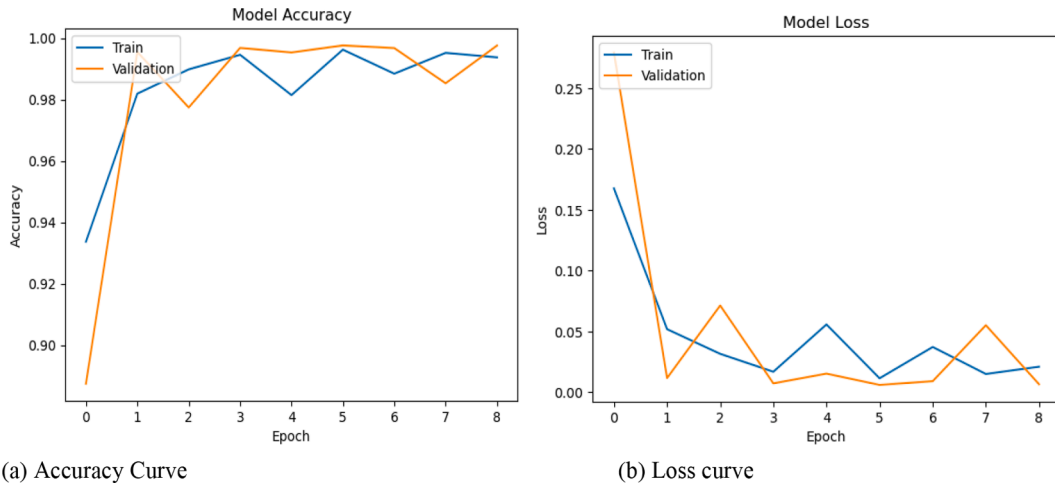


Fig. 9. (a) graphical representation of training and validation accuracy, (b) graphical representation of training and validation loss.

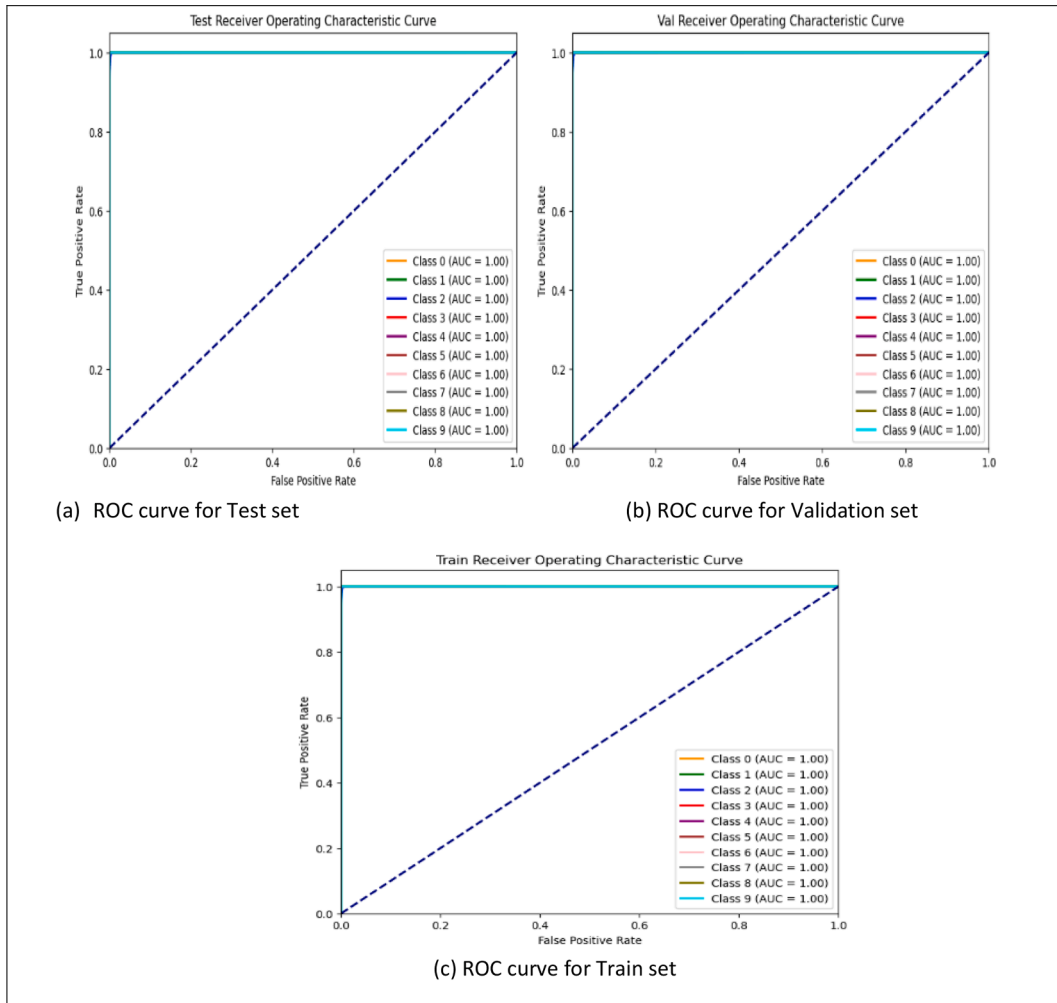


Fig. 10. ROC curve for the proposed model: based on the calculated TPR & FPR.

MCC allows us to measure the stability and robustness of our classification approach. The MCC value ranges from -1 to $+1$, where -1 signifies completely inaccurate predictions, 0 represents random predictions, and $+1$ indicates precise predictions. By calculating MCC for our multiclass classification problem, we are able to assess the model's capability to correctly classify instances across multiple classes,

encompassing both positive and negative predictions.

$$MCC = \frac{(TP*TN) - (FP*FN)}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (22)$$

Our proposed model achieves an impressive MCC value of 0.99707 , indicating a high degree of agreement between the predicted class labels

Table 9
Performance Evaluation Metrics of Proposed Model on Different Attack Classes.

Parameters/ Attack Classes	Precision										Recall										F1- score										Support									
	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9										
Train set	0	1	0.98	0.99	1	1	1	1	1	1	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9								
Validation	1	0.98	0.99	1	1	1	1	1	1	1	1	1	1	0.99	0.97	1	1	1	1	1	1	1	1	34,679	41,731	20,405	20,916	73,861	71,598	95,402	85,859	166,317	57,541							
set	1	0.98	0.99	1	1	1	1	1	1	1	1	1	1	0.99	0.95	1	1	1	1	1	1	1	1	4889	5981	2897	2981	10,624	10,301	10,799	12,150	23,889	8105							
Test set	1	0.97	0.99	1	1	1	1	1	1	1	1	1	1	0.98	0.96	1	1	1	1	1	1	1	1	9980	12,006	5766	5952	21,389	20,296	21,484	24,564	47,459	16,336							

and the actual ground truth across the various classes. This suggests that our model excels in effectively distinguishing and accurately classifying instances into their respective categories, resulting in a notably high level of prediction accuracy. The obtained MCC value not only demonstrates the model's strong performance, but it also serves as a testament to its robustness and stability, reaffirming its reliability in multiclass classification tasks.

In summary, our proposed model (AttackNet) has showcased outstanding performance in effectively classifying various types of botnet attacks. It has consistently demonstrated elevated precision, recall, and F1 scores across all attack classes, attaining a commendable equilibrium between accuracy and the capability to identify pertinent instances. The confusion matrix's elucidation and additional metrics offered comprehensive insights into the model's performance and streamlined optimization endeavors. Overall, our proposed model stands as a robust solution, excelling in the nuanced task of botnet attack classification.

4.5. Ablation study of proposed model

This section incorporates an ablation study aimed at providing comprehensive insights into the constituent elements of the model and their respective contributions. The assessment encompasses the following scenarios:

- Case A: Training the dataset using a 1-D Convolutional Neural Network (CNN).
- Case B: Training the dataset utilizing a 2-D Convolutional Neural Network (CNN).
- Case C: Training the dataset employing a Gated Recurrent Unit (GRU).
- Case D: Training the dataset using the proposed model.

This systematic evaluation of distinct cases serves to dissect the influence and efficacy of each model variant, thereby facilitating a nuanced understanding of their individual roles and impacts on overall performance.

Fig. 18 delineates the discernible effects resulting from each implemented enhancement. The ascending trends observed in accuracy, precision, and recall scores underscore the evident advantages derived from the refinement of the proposed model, offering profound insights into its constituent elements. The Table 11 gives a detailed breakdown of scores from our ablation study. It’s important to note that Case D, where we trained the dataset using our proposed model, performed better than all other cases. This emphasizes that our proposed model is more effective compared to other setups, showing better results across various evaluation measures.

5. Findings and Comparison with benchmark

5.1. Comparison with state of art work

This section compares the proposed model AttackNet to various cutting-edge strategies for detecting IoT botnet attacks using the “Detection of IoT botnet attacks N_BaIoT” security dataset. Table 12 shows the performance values for these current approaches on various parameters. In terms of accuracy, recall, and loss, the proposed AttackNet model outperforms the other models for multiclass classification of botnet attacks. The DNN-LSTM model (Hasan T et al.) achieved higher accuracy, precision, recall, and F1 score compared to the proposed AttackNet model. However, the difference is relatively small and both demonstrate high performance. The number of classes in proposed AttackNet model is higher (10 classes) compared to the DNN-LSTM model (6 classes). The proposed AttackNet model also provides detailed loss information, which is valuable for assessing the training performance. The proposed approach’s improved performance is due to

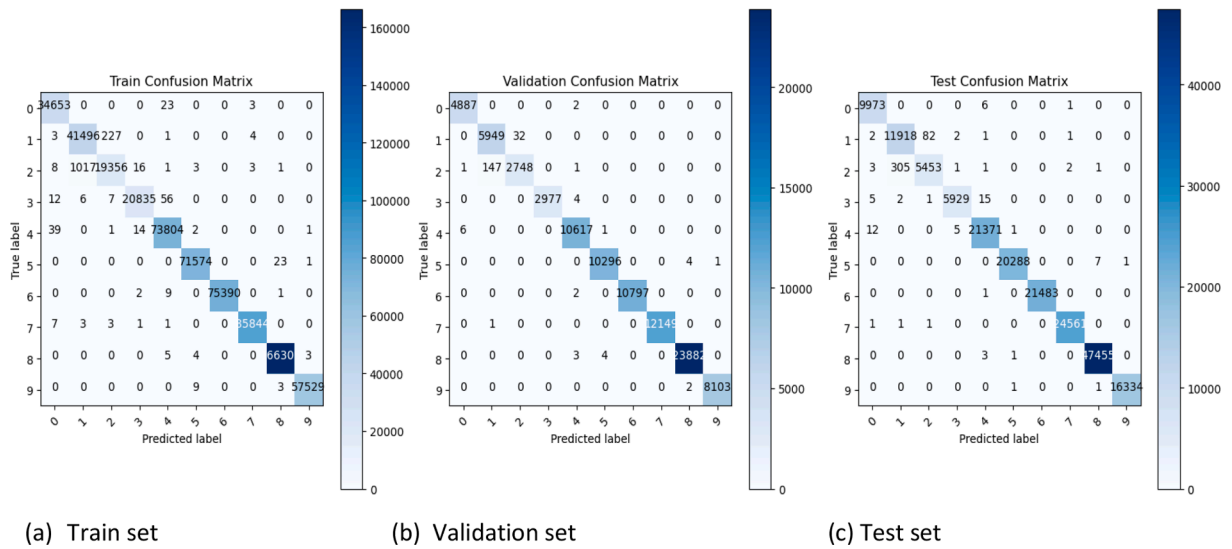


Fig. 11. Confusion matrix of the proposed model on the different target classes (a) for the train set (b) for the validation set (c) for the test set.

Table 10

Class-wise Performance Metrics Analysis.

Classes	True negativrate	Negative predictivevalue	False positivrate	False negativrate	False discoveryrate	False omissionrate
0	0.999869	0.999960	0.000131	0.000701	0.002301	0.000040
1	0.998222	0.999491	0.001778	0.007330	0.025192	0.000509
2	0.999532	0.998258	0.000468	0.054284	0.015171	0.001742
3	0.999955	0.999872	0.000045	0.003864	0.001347	0.000128
4	0.999835	0.999890	0.000165	0.000842	0.001262	0.000110
5	0.999982	0.999951	0.000018	0.000394	0.000148	0.000049
6	1.000000	0.999994	0.000000	0.000047	0.000000	0.000006
7	0.999975	0.999981	0.000025	0.000122	0.000163	0.000019
8	0.999935	0.999971	0.000065	0.000084	0.000190	0.000029
9	0.999994	0.999988	0.000006	0.000122	0.000061	0.000012

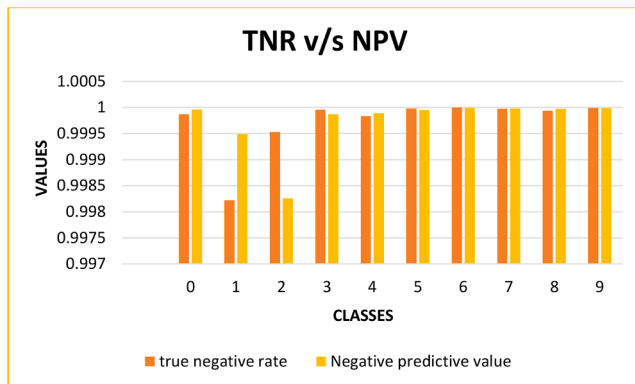


Fig. 12. TNR vs NPV: Analyzing Performance Measures.

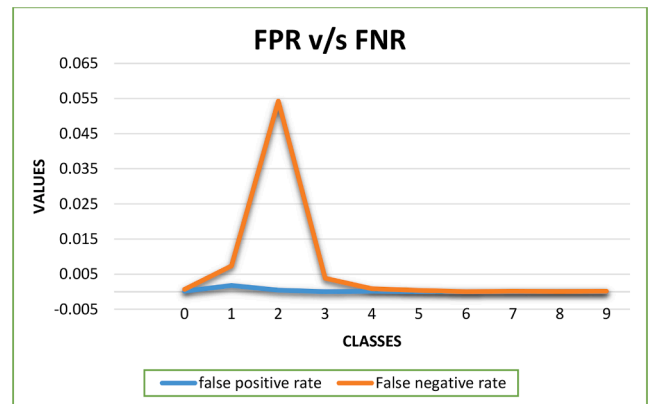


Fig. 13. FPR vs FNR: Analyzing Performance Measures.

its two-phase method. Significant features that have an impact on the training data are picked while scaling the dataset to normalize it, and redundant features are removed. The Conv 1D layer gathers local features in the second phase, while the GRU layer captures global characteristics by taking the entire sequence into account. By collecting both low-level and high-level features from the input sequence, the model is able to accurately categories data using these two strategies. Deep learning-friendly non-linear activation functions such as Relu and softmax are used in the model. To generate the final class probabilities, relu is employed in the Conv 1D and dense layers and softmax in the output layer. The effectiveness of both phases justifies the proposed model's accuracy in predicting various botnet attack types.

This section provides informative comparisons between the performance of the proposed model and current cutting-edge IoT botnet attack detection approaches. Fig. 19 compares the accuracy parameter of the proposed model to the technique currently in use. Overall, the results reveal that our proposed model AttackNet outperforms other strategies and correctly classifies various botnet attack types.

5.2. Complexity analysis

5.2.1. Time Complexity

The time complexity of the above model can be approximated by the

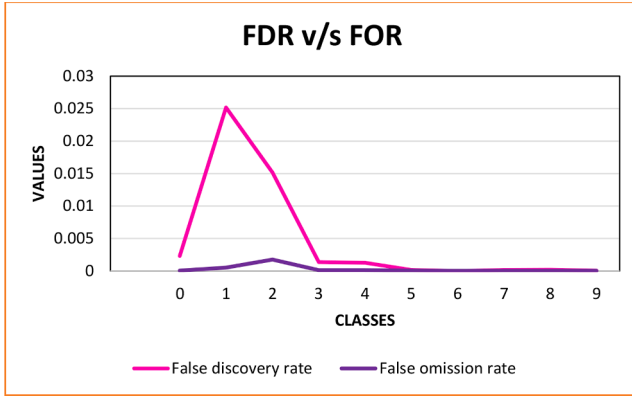


Fig. 14. FDR vs FOR: Analyzing Performance Measures.

number of trainable parameters in the model, which is a function of several hyperparameters such as the number of filters in the convolutional layers, the kernel size of the convolutional filters, the number of units in the GRU layers, and the number of units in the dense layers. The number of trainable parameters in the model can be computed as follows:

$$\text{num_params} = (\text{num_filters} * \text{kernel_size} + 1) * \text{num_features} + \text{num_features}^2 + \text{num_features} * \text{num_timesteps} * (\text{num_timesteps} - 1) * \text{recurrent_units} + \text{num_features} * \text{dense_units} + \text{dense_units} + \text{num_classes}$$

where:

- num_filters: the number of filters in the convolutional layers
- kernel_size: the size of the convolutional kernels

- num_features: the number of input features
- num_timesteps: the number of time steps in the input sequence
- recurrent_units: the number of units in the GRU layers
- dense_units: the number of units in the dense layers
- num_classes: the number of output classes

The time complexity of the model can be approximated as proportional to the number of trainable parameters. This is because the most computationally expensive part of training the model is computing the gradients of the loss function with respect to the model parameters, which requires evaluating the model at each training step.

The number of training steps required to train the model is proportional to the number of epochs and the size of the training data. Therefore, the time complexity of the model is also dependent on these factors. Table 13 shows the computational time complexity analysis on the dataset.

5.2.2. Space Complexity

The space complexity of the model is also proportional to the number of trainable parameters in the model. This is because each trainable parameter requires a certain amount of memory to store its value, and the total memory required to store all the trainable parameters is proportional to their number. In addition to the trainable parameters, the model also requires memory to store the input data, intermediate activations, and gradients during training. The memory required for these operations is proportional to the size of the input data, the number of units in the model, and the number of training steps required to train the model.

Overall, the time and space complexity of the model can be quite high, especially for large input data and a large number of trainable parameters. However, efficient algorithms and hardware accelerators

	benign	mirai_udp	gafgyt_combo	gafgyt_junk	gafgyt_scan	gafgyt_udp	mirai_ack	mirai_scan	mirai_syn	mirai_udpplain
benign	9973 5.38%	0 0.00%	0 0.00%	0 0.00%	6 0.00%	0 0.00%	0 0.00%	1 0.00%	0 0.00%	0 0.00%
mirai_udp	2 0.00%	11918 6.43%	82 0.04%	2 0.00%	1 0.00%	0 0.00%	0 0.00%	1 0.00%	0 0.00%	0 0.00%
gafgyt_combo	3 0.00%	305 0.16%	5453 2.94%	1 0.00%	1 0.00%	0 0.00%	0 0.00%	2 0.00%	1 0.00%	0 0.00%
gafgyt_junk	5 0.00%	2 0.00%	1 0.00%	5929 3.20%	15 0.01%	0 0.00%	0 0.00%	0 0.00%	0 0.00%	0 0.00%
gafgyt_scan	12 0.01%	0 0.00%	0 0.00%	5 0.00%	21371 11.54%	1 0.00%	0 0.00%	0 0.00%	0 0.00%	0 0.00%
gafgyt_udp	0 0.00%	0 0.00%	0 0.00%	0 0.00%	0 0.00%	20288 10.95%	0 0.00%	0 0.00%	7 0.00%	1 0.00%
mirai_ack	0 0.00%	0 0.00%	0 0.00%	0 0.00%	1 0.00%	0 0.00%	21483 11.60%	0 0.00%	0 0.00%	0 0.00%
mirai_scan	1 0.00%	1 0.00%	1 0.00%	0 0.00%	0 0.00%	0 0.00%	0 0.00%	24561 13.26%	0 0.00%	0 0.00%
mirai_syn	0 0.00%	0 0.00%	0 0.00%	0 0.00%	3 0.00%	1 0.00%	0 0.00%	0 0.00%	47455 25.62%	0 0.00%
mirai_udpplain	0 0.00%	0 0.00%	0 0.00%	0 0.00%	0 0.00%	1 0.00%	0 0.00%	0 0.00%	1 0.00%	16334 8.82%

Fig. 15. Class wise confusion matrix of proposed model on the different target classes.

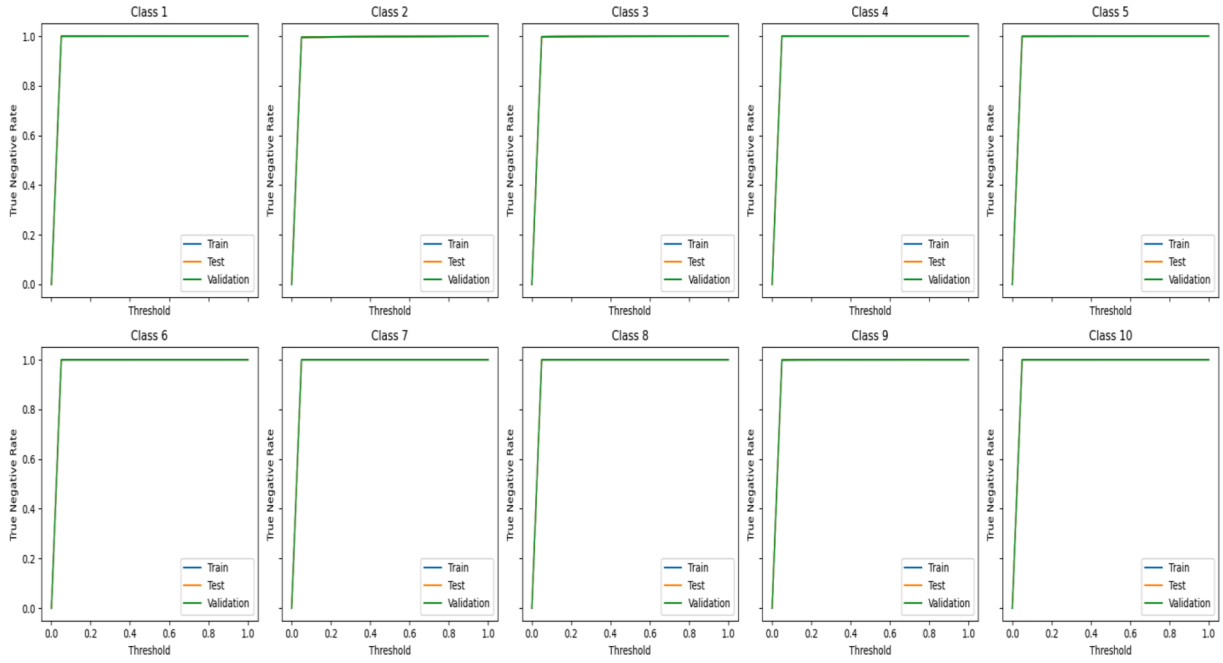


Fig. 16. TNR values at each class on train, validation and test sets for correctly classified negative instances.

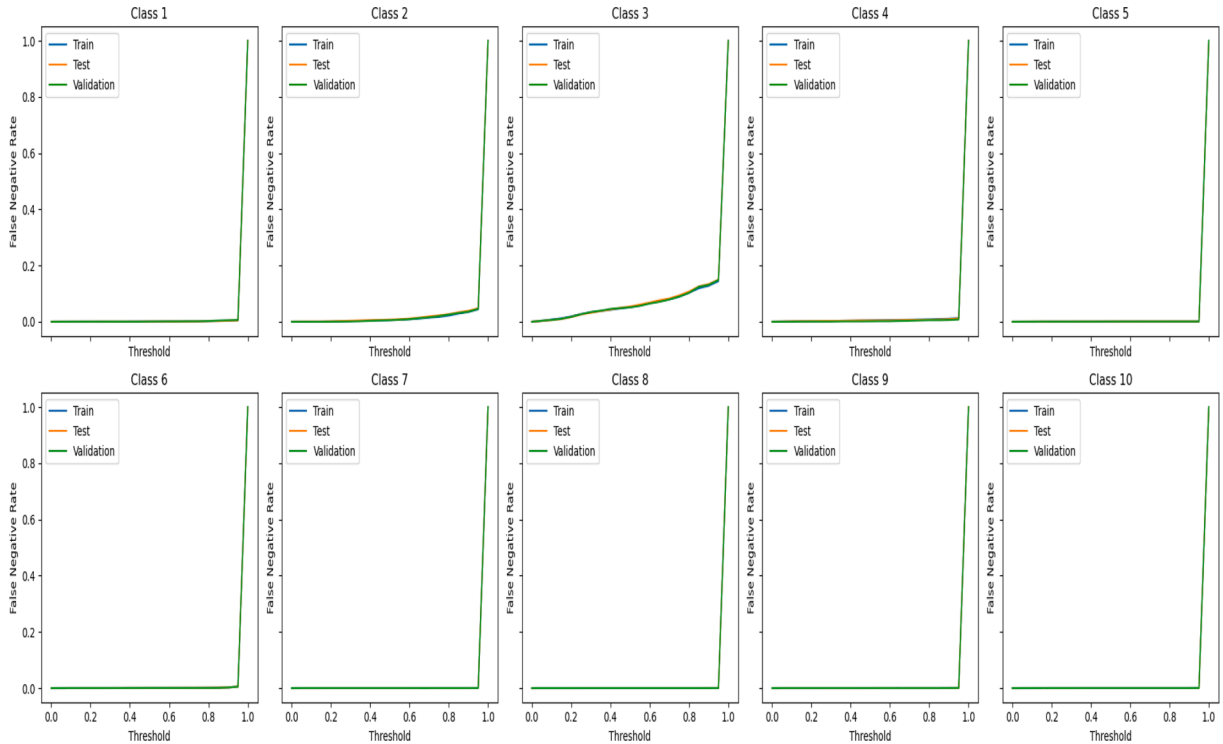


Fig. 17. FNR values at each class on train, validation and test sets for incorrectly classified positive instances.

such as GPUs can help to mitigate these issues. Additionally, techniques such as weight regularization and early stopping can help to reduce the number of trainable parameters and reduce overfitting, which can also help to reduce the time and space complexity of the model.

5.3. Findings

i. i Reliability and Trustworthiness:

To evaluate the reliability of our model, we consider a scenario where our model consists of 10 distinct attack types or learners. Due to the diverse nature of Deep learning, the errors that arise in these various attacks are independent and unrelated to each other. Even if certain learners exhibit inaccuracies, the remaining learners may still demonstrate accuracy, thereby allowing our approach to effectively classify intrusion attacks in Deep learning-based Industrial Internet of Things (IIoT) networks.

Fig. 20 shows that the error rate of the learner for 10 different attack

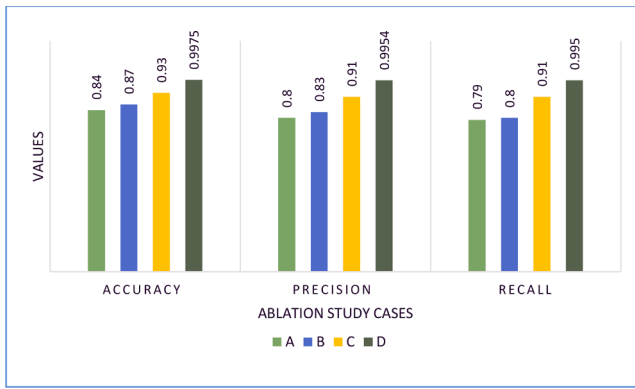


Fig. 18. Ablation study result on N_BaIoT dataset.

Table 11

Ablation study score for the proposed model AttackNet.

Ablation study cases	Cases	Accuracy	Precision	Recall
Train the dataset with 1D CNN	A	0.84	0.80	0.79
Train the dataset with 2D CNN	B	0.87	0.83	0.80
Train the dataset with GRU	C	0.93	0.91	0.91
Train the dataset with Proposed model	D	0.9975	0.9954	0.9950

types is less than 0.006779. This means that our method is effective at detecting attacks in IIoT networks. We verified the trustworthiness of our model by classifying attacks with one dataset and found that the error rate was low.

Fig. 21 shows the accuracy of the proposed model using a hybrid approach on 10 different attack types. The accuracy of the model is higher for some attack types than others, but overall the model is effective at detecting attacks.

ii. Dependability performance analysis:

Table 12

Comparative Analysis of Proposed Model and State-of-the-Art Approaches on the 'N_BaIoT' IoT Botnet Attack Detection Dataset.

Reference & year	Purpose	Model name	Classification type	Side	Accuracy	Precision	Recall	F1-score	Loss
2021,(Baby R et al.,)	IoT attack detection using deep reinforcement learning	DNN-DRL	Binary classification	Client	83.45	–	–	–	–
2022,(CU OK et al.,)	Attack detection using different autoencoder	Variational auto-encoder	Binary classification	Client	96.156	–	–	–	–
2021,(Siddiqui AJ et al.,)	TempCode-IoT model for detection of IDS flow	TempCode-IoT(flow-based statistical feature model)	Binary classification	Client	–	98.4	99.4	98.9	–
2022,(Shafiq U et al.,)	Anomaly detection using transfer learning for DDOS attack	Auto-encoder	Multiclass classification(3 classes)	Client	99.54	–	–	–	–
2022,(Cunha AA et al.,)	Botnet classification in the Internet of Things	CNN	Multiclass classification (3 classes)	Client	0.98	0.98	0.97	0.976	–
2021,(Alharbi A et al.,)	Botnet attack detection using bio-inspired algorithm	Local-global best bat algorithm for neural network(LGBA-NN)	Multiclass classification (10 classes)	Client	0.90	0.90	0.90	0.90	0.2
2021,(Hezam AA et al.,)	Deep learning approach for botnet attack	RNN	Multiclass classification	Client	89.75	–	–	–	–
2021,(Parra GD et al.,)	Hybrid model to detect the botnet attack in IoT application	CNN-LSTM	Multiclass classification	Client	93	94	89	85	0.13
2022,(Hasan T et al.,)	Hybrid deep learning approach for botnet attack in securing industrial IoT	DNN_LSTM	Multiclass Classification(6 classes)	Client	99.94	99.91	99.86	99.86	–
2023,(Haq MA et al.,)	Deep learning prediction model	DBoTM	Regression problem	Client	R ² = 71 %	–	–	–	–
Our proposed model	Hybrid-deep learning model for IDS in IIoT environment	AttackNet (CNN + GRU)	Multiclass classification (On 10 classes)	Client-side	99.75	99.54	99.50	99.52	0.0063

In this section, we analyze the dependability performance of our proposed model, which encompasses availability, efficiency, and scalability. We carefully select features and apply our model to accurately classify both normal and attack scenarios. This ensures that our model is always available and performs well. We evaluate our model using metrics such as accuracy, precision, recall, F1 score, and roc. Our model outperforms several existing approaches and achieves this improved performance with minimal computational loss. This is illustrated in

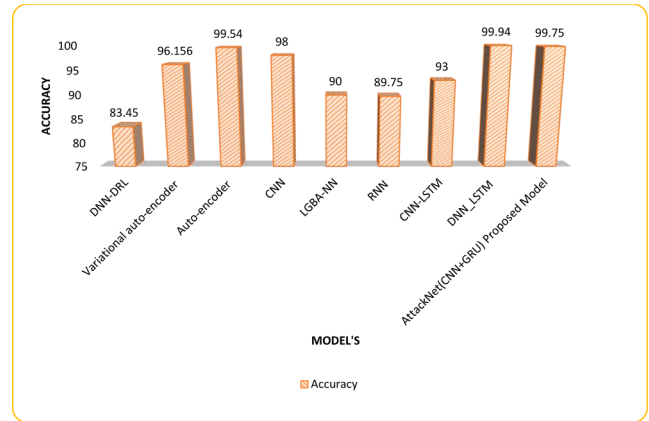


Fig. 19. Graphical representation of the proposed model with the existing state of artwork on the accuracy parameter.

Table 13

Computational Time Analysis: Training, Validation, and Testing Performance on the Dataset.

Dataset	Computational Time
Train set	160 s 8 ms/step
Validation set	23 s 8 ms/step
Test set	45 s 8 ms/step

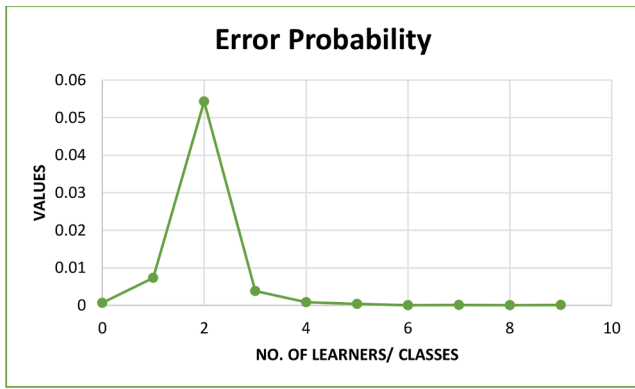


Fig. 20. Simulated error probability of each class.

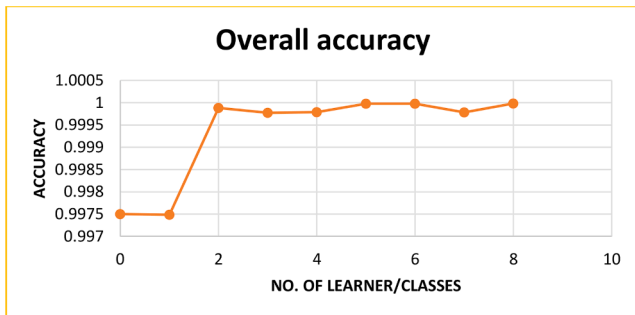


Fig. 21. Overall accuracy of each classes of our proposed model (AttackNet).

Fig. 9.

Finally, we have observed enhanced scalability characteristics in our proposed model by incorporating diverse trusted data sources from various IoT devices into the training dataset. This inclusion ensures maximum consistency and a wider range of data. Remarkably, as we increased the epoch number from 10 to 100, the accuracy of our proposed model remained nearly unchanged, indicating its scalability. The Fig. 22 illustrates the scalability performance of our proposed model.

6. Conclusion and future work

The Internet of Things (IoT) has experienced significant growth in recent years, leading to an increase in traffic data and the need for a reliable and dependable intrusion detection system to secure it. However, the lack of labeled information and high-dimensional data makes it difficult to develop effective intrusion detection systems for IoT networks. This paper proposes a hybrid deep learning approach using a novel CNN-GRU model named AttackNet to detect various botnet

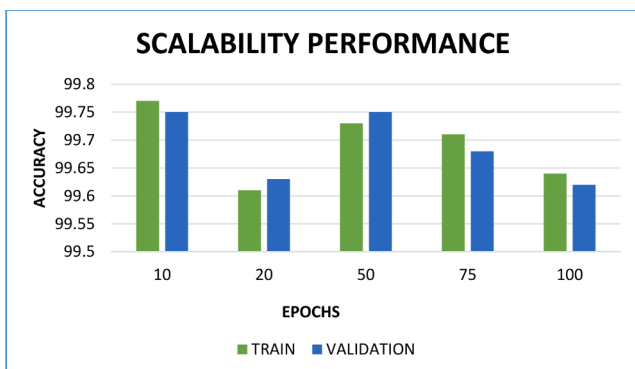


Fig. 22. Scalability analysis of our proposed model (AttackNet).

attacks in the IoT environment. The model is trained and tested on the "Detection of IoT botnet attacks N_BaIoT" security dataset, demonstrating its superiority over other deep learning methods with an accuracy of 99.75 % and a validation loss of 0.0063 at a 0.001 learning rate. Extensive experiments were conducted to evaluate the model's strength in various scenarios, indicating that hybrid deep learning approaches can improve accuracy, precision, recall, and F1 score for IoT botnet attack detection. The proposed AttackNet model operates transformation at all levels of the CNN's encoding layer to improve the effectiveness of the transferring process, making it meaningful even when there is label information in the source domain but not in the target domain.

In the future, the proposed framework can be enhanced to detect other IoT-centric attacks and extended to a real IoT network architecture. The performance of AttackNet can also be investigated over a more extensive and combined dataset of different attack types to tackle a more challenging multiclass anomaly detection problem effectively.

6.1. Limitations

While AttackNet excels in IoT botnet attack detection, its reliance on labelled data and specific attack types may limit generalization. The model's performance may vary with evolving attack strategies, necessitating continuous adaptation. Additionally, the scalability and real-world applicability of the proposed framework require further exploration for comprehensive IoT security solutions.

6.2. Potential applications

Following are the Potential application of the proposed model AttackNet:

- **Enhanced security for IIoT networks:** AttackNet can be deployed in critical IIoT infrastructure to protect against sophisticated botnet attacks, ensuring system reliability and preventing disruptions.
- **Improved cyber defense capabilities:** The proposed model can be integrated into existing security solutions to strengthen cyber defense capabilities and enhance threat detection effectiveness.
- **Real-time monitoring and analysis:** AttackNet's real-time capabilities enable continuous monitoring and analysis of network traffic, facilitating the identification of potential threats and providing valuable insights for security analysts.

CRedit authorship contribution statement

Himanshu Nandanwar: Conceptualization, Data curation, Writing – original draft, Writing – review & editing, Visualization, Methodology.
Rahul Katarya: Supervision, Formal analysis, Editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

- Abomhara, M., & Kojen, G. M. (2015). Cyber security and the internet of things: Vulnerabilities, threats, intruders and attacks. *Journal of Cyber Security and Mobility*, 65–88.
- Abusitta, A., de Carvalho, G. H., Wahab, O. A., Halabi, T., Fung, B. C., & Al Mamoori, S. (2023). Deep learning-enabled anomaly detection for IoT systems. *Internet of Things*, 21, Article 100656.

- Aldweesh, A., Derhab, A., & Emam, A. Z. (2020). Deep learning approaches for anomaly-based intrusion detection systems: A survey, taxonomy, and open issues. *Knowledge-Based Systems*, 189, Article 105124.
- Alharbi, A., Alosaimi, W., Alyami, H., Rauf, H. T., & Damaševićius, R. (2021). Botnet attack detection using local global best bat algorithm for industrial internet of things. *Electronics*, 10(11), 1341.
- Arachchige, P. C. M., Bertok, P., Khalil, I., Liu, D., Camtepe, S., & Atiquzzaman, M. (2020). A trustworthy privacy preserving framework for machine learning in industrial IoT systems. *IEEE Transactions on Industrial Informatics*, 16(9), 6092–6102.
- Awotunde, J. B., Ayo, F. E., Panigrahi, R., Garg, A., Bhoi, A. K., & Barsocchi, P. (2023). A multi-level random Forest model-based intrusion detection using fuzzy inference system for internet of things networks. *International Journal of Computational Intelligence Systems*, 16(1), 31.
- Baby, R., Pooranian, Z., Shojafar, M., & Tafazolli, R. (2023). In May. *A Heterogenous IoT Attack Detection Through Deep Reinforcement Learning: A Dynamic ML Approach* (pp. 479–484). IEEE.
- Balan, K., Abdulrazzak, L. F., Khan, A. S., Julaihi, A. A., Tarmizi, S., Pillay, K. S., & Sallehudin, H. (2018). RSSI and public key infrastructure based secure communication in autonomous vehicular networks. *International Journal of Advanced Computer Science and Applications*, 9(12).
- Basil, N., Marhoon, H. M., & Ibrahim, A. R. (2023). A new thrust vector-controlled rocket based on JOA using MCDA. *Measurement: Sensors*, 26, Article 100672.
- Basil, N., Marhoon, H. M., Gokulakrishnan, S., & Buddhi, D. (2022). Jaya optimization algorithm implemented on a new novel design of 6-DOF AUV body: A case study. *Multimedia Tools and Applications*, 1–26.
- Bhandari, G., Lyth, A., Shalaginov, A., & Grønli, T. M. (2023). Distributed deep neural-network-based Middleware for cyber-attacks detection in Smart IoT ecosystem: A novel framework and performance evaluation approach. *Electronics*, 12(2), 298.
- Cunha, A. A., Borges, J. B., & Loureiro, A. A. (2022, October). Classification of botnet attacks in IoT using a convolutional neural network. In *In Proceedings of the 18th ACM International Symposium on QoS and Security for Wireless and Mobile Networks* (pp. 63–70).
- CU, O. K., Pranavi, D., Laxmi, B. A., & Devasena, R. (2022). Variational Autoencoder for IoT Botnet Detection. In *Using Computational Intelligence for the Dark Web and Illicit Behavior Detection* (pp. 74–88). IGI Global.
- Diro, A. A., & Chilamkurti, N. (2018). Distributed attack detection scheme using deep learning approach for internet of things. *Future Generation Computer Systems*, 82, 761–768.
- Elrawy, M. F., Awad, A. I., & Hamed, H. F. (2018). Intrusion detection systems for IoT-based smart environments: A survey. *Journal of Cloud Computing*, 7(1), 1–20.
- Ferrag, M. A., et al. (2022). "Edge-IoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning." *IEEE Access*, 10, 40281–40306.
- Ferrag, M. A., Maglaras, L., Moschogiannis, S., & Janicke, H. (2020). Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study. *Journal of Information Security and Applications*, 50, Article 102419.
- Frailie, F., Tagawa, T., Poler, R., & Ortiz, A. (2018). Trustworthy industrial IoT gateways for interoperability platforms and ecosystems. *IEEE Internet of Things Journal*, 5(6), 4506–4514.
- Garcia, S., Parmisano, A., & Erquiaga, M. J. (2020). IoT-23: A labeled dataset with malicious and benign IoT network traffic. Stratosphere Lab., Praha, Czech Republic, Tech. Rep.
- Hasan, T., Malik, J., Bibi, I., Khan, W. U., Al-Wesabi, F. N., Dev, K., & Huang, G. (2022). Securing industrial internet of things against botnet attacks using hybrid deep learning approach. *IEEE Transactions on Network Science and Engineering*.
- Hassan, R., Qamar, F., Hasan, M. K., Aman, A. H. M., & Ahmed, A. S. (2020). Internet of things and its applications: A comprehensive survey. *Symmetry*, 12(10), 1674.
- Haq, M. A. (2023). DBoTPM: A deep neural network-based botnet prediction model. *Electronics*, 12(5), 1159.
- Heidari, A., Navimipour, N. J., & Unal, M. (2023). A secure intrusion detection platform using blockchain and radial basis function neural networks for internet of drones. *IEEE Internet of Things Journal*.
- Hezam, A. A., Mostafa, S. A., Raml, A. A., Mahdin, H., & Khalaf, B. A. (2021, August). Deep learning approach for detecting botnet attacks in IoT environment of multiple and heterogeneous sensors. In *International Conference on Advances in Cyber Security* (pp. 317–328). Singapore: Springer Singapore.
- Ibrahim, A. R., Basil, N., & Mahdi, M. I. (2021). Implementation enhancement of AVR control system within optimization techniques. *International Journal of Nonlinear Analysis and Applications*, 12(2).
- Ioulianou, P., Vasilakis, V., Moscholios, I., & Logothetis, M. (2018). A signature-based intrusion detection system for the internet of things. *Information and Communication Technology Form*.
- Iqbal, R., Maniak, T., Doctor, F., & Karyotis, C. (2019). Fault detection and isolation in industrial processes using deep learning approaches. *IEEE Transactions on Industrial Informatics*, 15(5), 3077–3084.
- Jo, W., Kim, S., Lee, C., & Shon, T. (2020). Packet preprocessing in CNN-based network intrusion detection system. *Electronics*, 9(7), 1151.
- Jothi, B., & Pushpalatha, M. (2023). WLS-TRS—A novel optimized deep learning based intrusion detection framework for IoT networks. *Personal and Ubiquitous Computing*, 27(3), 1285–1301.
- Kaur, S., & Singh, M. (2020). Hybrid intrusion detection and signature generation using deep recurrent neural networks. *Neural Computing and Applications*, 32, 7859–7877.
- Khan, F., Alturki, R., Rahman, M. A., Mastorakis, S., Razzak, I., & Shah, S. T. (2022). Trustworthy and reliable deep-Learning-based cyberattack detection in industrial IoT. *IEEE Transactions on Industrial Informatics*, 19(1), 1030–1038.
- Khanday, S. A., Fatima, H., & Rakesh, N. (2023). Implementation of intrusion detection model for DDoS attacks in lightweight IoT networks. *Expert Systems with Applications*, 215, Article 119330.
- Koroniotis, N., Moustafa, N., Sitnikova, E., & Turnbull, B. (2019). Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset. *Future Generation Computer Systems*, 100, 779–796.
- Kumar, P., Kumar, R., Srivastava, G., Gupta, G. P., Tripathi, R., Gadekallu, T. R., & Xiong, N. N. (2021). PPSF: A privacy-preserving and secure framework using blockchain-based machine-learning for IoT-driven smart cities. *IEEE Transactions on Network Science and Engineering*, 8(3), 2326–2341.
- Li, B., Wu, Y., Song, J., Lu, R., Li, T., & Zhao, L. (2020). DeepFed: Federated deep learning for intrusion detection in industrial cyber-physical systems. *IEEE Transactions on Industrial Informatics*, 17(8), 5615–5624.
- Liu, Y., Wang, L., Li, D., & Wang, K. (2023). State-of-health estimation of lithium-ion batteries based on electrochemical impedance spectroscopy: A review. *Protection and Control of Modern Power Systems*, 8(1), 41.
- Lv, Z., Qiao, L., Li, J., & Song, H. (2020). Deep-learning-enabled security issues in the internet of things. *IEEE Internet of Things Journal*, 8(12), 9531–9538.
- Ma, N., Yin, H., & Wang, K. (2023). Prediction of the remaining useful life of supercapacitors at different temperatures based on improved long short-term memory. *Energies*, 16(14), 5240.
- Makkar, A., Garg, S., Kumar, N., Hossain, M. S., Ghoneim, A., & Alrashoud, M. (2020). An efficient spam detection technique for IoT devices using machine learning. *IEEE Transactions on Industrial Informatics*, 17(2), 903–912.
- Marhoon, H. M., Ibrahim, A. R., & Basil, N. (2021). Enhancement of electro hydraulic position servo control system utilising ant lion optimiser. *International Journal of Nonlinear Analysis and Applications*, 12(2), 2453–2461.
- Marhoon, H. M., Qasem, N., Basil, N., & Ibrahim, A. R. (2022). Design and simulation of a compact metal-graphene frequency reconfigurable microstrip patch antenna with FSS superstrate for 5G applications. *International Journal on Engineering Applications (IREA)*, 10(3), 193–201.
- Mehedi, S. T., Anwar, A., Rahman, Z., Ahmed, K., & Islam, R. (2022). Dependable intrusion detection system for IoT: A deep transfer learning based approach. *IEEE Transactions on Industrial Informatics*, 19(1), 1006–1017.
- Meidan, Y., Bohadana, M., Mathov, Y., Mirsky, Y., Shabtai, A., Breitenbacher, D., & Elovici, Y. (2018). N-baiot—network-based detection of IoT botnet attacks using deep autoencoders. *IEEE Pervasive Computing*, 17(3), 12–22.
- Mohammed, Y. R., Basil, N., Bayat, O., & Mohammed, A. H. (2020). A new novel optimization techniques implemented on the AVR control system using MATLAB-SIMULINK.
- Mohamadwasel, N. B. (2020, November). Rider Optimization Algorithm implemented on the AVR Control System using MATLAB with FOPID. In *IOP Conference Series: Materials Science and Engineering* (Vol. 928, No. 3, p. 032017). IOP Publishing.
- Mohanta, B. K., Jena, D., Satapathy, U., & Patnaik, S. (2020). Survey on IoT security: Challenges and solution using machine learning, artificial intelligence and blockchain technology. *Internet of Things*, 11, Article 100227.
- Moustafa, N. (2017). *Designing an online and reliable statistical anomaly detection framework for dealing with large high-speed network traffic*. UNSW Sydney. Doctoral dissertation.
- Oseni, A., Moustafa, N., Creech, G., Sohrabi, N., Strelzoff, A., Tari, Z., & Linkov, I. (2022). An explainable deep learning framework for resilient intrusion detection in IoT-enabled transportation networks. *IEEE Transactions on Intelligent Transportation Systems*, 24(1), 1000–1014.
- Parra, G. D. L. T., Rad, P., Choo, K. K. R., & Beebe, N. (2020). Detecting internet of things attacks using distributed deep learning. *Journal of Network and Computer Applications*, 163, Article 102662.
- Qiu, C., Yu, F. R., Yao, H., Jiang, C., Xu, F., & Zhao, C. (2018). Blockchain-based software-defined industrial internet of things: A dueling deep $\{q\}$ -learning approach. *IEEE Internet of Things Journal*, 6(3), 4627–4639.
- Raheem, F. S., & Basil, N. (2023). Automation intelligence photovoltaic system for power and voltage issues based on black hole optimization algorithm with FOPID. *Measurement: Sensors*, 25, Article 100640.
- Sengupta, J., Ruj, S., & Bit, S. D. (2020). A comprehensive survey on attacks, security issues and blockchain solutions for IoT and IIoT. *Journal of Network and Computer Applications*, 149, Article 102481.
- Sha, K., Wei, W., Yang, T. A., Wang, Z., & Shi, W. (2018). On security challenges and open issues in internet of things. *Future generation computer systems*, 83, 326–337.
- Shafiq, U., Shahzad, M. K., Anwar, M., Shaheen, Q., Shiraz, M., & Gani, A. (2022). Transfer Learning Auto-Encoder Neural Networks for Anomaly Detection of DDoS Generating IoT Devices. *Security and Communication Networks*, 2022.
- Sharafaldin, I., et al. (2019). Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy. *2019 International Carnahan Conference on Security Technology (ICCST)*. IEEE.
- Sharma, M., Pant, S., Yadav, P., Sharma, D. K., Gupta, N., & Srivastava, G. (2023). Advancing security in the industrial internet of things using deep progressive neural networks. *Mobile Networks and Applications*, 1–13.
- Shevchik, S. A., Masinelli, G., Kenel, C., Leinenbach, C., & Wasmer, K. (2019). Deep learning for in situ and real-time quality monitoring in additive manufacturing using acoustic emission. *IEEE Transactions on Industrial Informatics*, 15(9), 5194–5203.
- Siddiqui, A. J., & Boukerche, A. (2021). TempoCode-IoT: Temporal codebook-based encoding of flow features for intrusion detection in internet of things. *Cluster Computing*, 24, 17–35.
- Stolfo, S. J., Fan, W., Lee, W., Prodromidis, A., & Chan, P. K. (2000, January). Cost-based modeling for fraud and intrusion detection: Results from the JAM project. In *Proceedings DARPA Information Survivability Conference and Exposition. DISCEX'00* (Vol. 2, pp. 130–144). IEEE.

- Tang, J., Sun, D., Liu, S., & Gaudiot, J. L. (2017). Enabling deep learning on IoT devices. *Computer*, 50(10), 92–96.
- Tavallae, M., et al. (2009). A detailed analysis of the KDD CUP 99 data set. *2009 IEEE symposium on computational intelligence for security and defense applications*. Ieee.
- Wang, S., Xu, W., & Liu, Y. (2023). Res-TranBiLSTM: An intelligent approach for intrusion detection in the internet of things. *Computer Networks*, 235, Article 109982.
- Yao, R., Wang, N., Liu, Z., Chen, P., & Sheng, X. (2021). Intrusion detection system in the advanced metering infrastructure: A cross-layer feature-fusion CNN-LSTM-based approach. *Sensors*, 21(2), 626.
- Yi, Z., Chen, Z., Yin, K., Wang, L., & Wang, K. (2023). Sensing as the key to the safety and sustainability of new energy storage devices. *Protection and Control of Modern Power Systems*, 8(1), 1–22.
- Yu, X., Shang, Y., Zheng, L., & Wang, K. (2023). Application of nanogenerators in the field of acoustics. *ACS Applied Electronic Materials*, 5(9), 5240–5248.
- Zhai, F., Yang, T., Chen, H., He, B., & Li, S. (2023). Intrusion detection method based on CNN-GRU-FL in a smart grid environment. *Electronics*, 12(5), 1164.
- Zikria, Y. B., Afzal, M. K., Kim, S. W., Marin, A., & Guizani, M. (2020). Deep learning for intelligent IoT: Opportunities, challenges and solutions. *Computer Communications*, 164, 50–53.



Rahul Katarya is currently working as a Professor at Delhi Technological University, Delhi, India, in the Department of Computer Science and Engineering. He has published more than 150 papers in refereed conferences and journals. His research interests include Data Science, Big Data analytics, Web Mining, Social Networks Analysis, Recommender Systems, Machine Learning, Data Mining, Human Behavior. Evolutionary Computing



Himanshu Nandanwar is a research scholar at Delhi Technological University, Delhi, India, in the Department of Computer Science and Engineering. He received his M. Tech & B. Tech degree in Information Technology from Delhi Technological University. His research interests are the Internet of things, Networking, Cloud computing IoT security, 5G/6G.